

Tecnologia de Comunicação

Módulo 5 – Rádio Definido por Software, Wi-Fi, OFDM e recepção IEEE 802.11

Da amostra I/Q capturada no SDR aos bits de um quadro Wi-Fi

Prof. Paulo Matias

Objetivos deste módulo

Ao fim desta aula, o aluno deve saber responder:

- O que é *Software Defined Radio* (SDR) e onde fica a fronteira entre RF analógico, conversores e DSP.
- Como um receptor SDR pode ser construído com blocos simples: filtros, misturadores, osciladores, ADCs e processamento I/Q.
- Por que Wi-Fi 802.11a/g usa *Orthogonal Frequency-Division Multiplexing* (OFDM), prefixo cíclico (CP), preâmbulo, treinamento e pilotos.
- Quais defeitos (*impairments*) o sinal sofre e qual bloco do receptor corrige cada um.
- Como cada transformação no código da prática modifica o sinal e por que ela funciona.
- Como funcionam códigos convolucionais binários (BCC), Viterbi com *soft-decision* e *Low-Density Parity-Check* (LDPC) com passagem de mensagens.

O que não vamos repetir do zero

Este módulo reaproveita conceitos já trabalhados:

- I/Q, PSK/QAM, constelações, BER, SNR e E_b/N_0 .
- AWGN, ruído térmico, figura de ruído e orçamento de enlace.
- CRC, GF(2), LFSR e diferença entre detecção e correção de erros.
- ISI, equalização, sincronismo de portadora, erro de fase/frequência e erro de temporização.
- FFT/IFFT já apareceram em outros contextos; aqui elas viram o coração do modem.

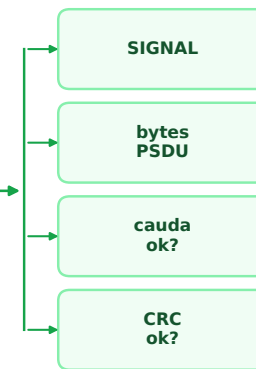
A novidade é integrar tudo em uma PHY Wi-Fi real: o sinal chega como amostras complexas e sai como bytes de um quadro MAC.

Prática 5: problema concreto

fontes de amostras I/Q



saídas e validações



A prática implementa um receptor 802.11a/g em Python:

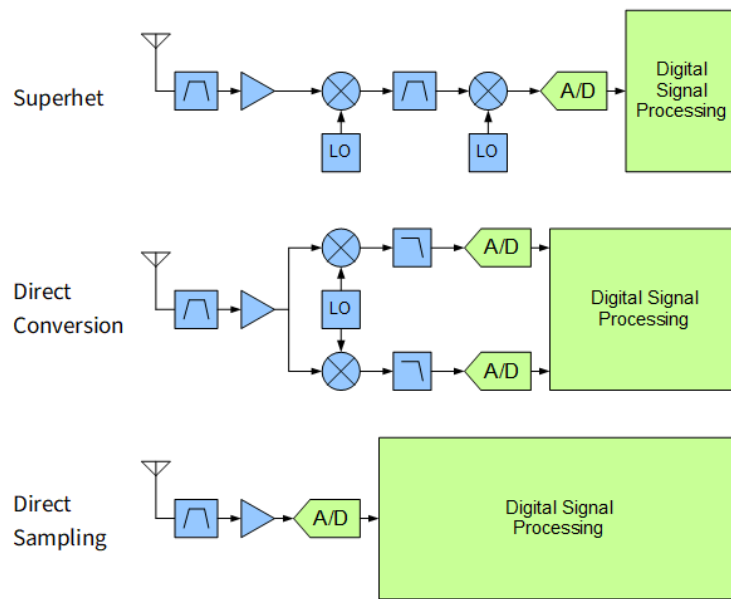
- Entrada: vetor complex64 com amostras I/Q em banda base.
- Saída: campo SIGNAL, dados MAC, verificação de cauda e CRC.
- Modo sintético: transmissor, modelo de defeitos, receptor e EVM (*Error Vector Magnitude*).
- Modo SDR: captura com LimeSDR/GNU Radio e processamento offline.

SDR: onde colocar o software?

SDR não significa “sem RF analógico”.

- A antena, filtros, LNA/VGA, misturadores e osciladores continuam existindo.
- O que muda é a fronteira: mais funções passam a ser parametrizadas ou implementadas em DSP.
- Quanto mais cedo se amostra, maior a taxa e a exigência do ADC.
- Quanto mais tarde se amostra, mais rígido fica o hardware analógico.

Em Wi-Fi, o receptor físico é quase sempre uma combinação: front-end RF dedicado + ADC/DAC + DSP/FPGA/CPU.



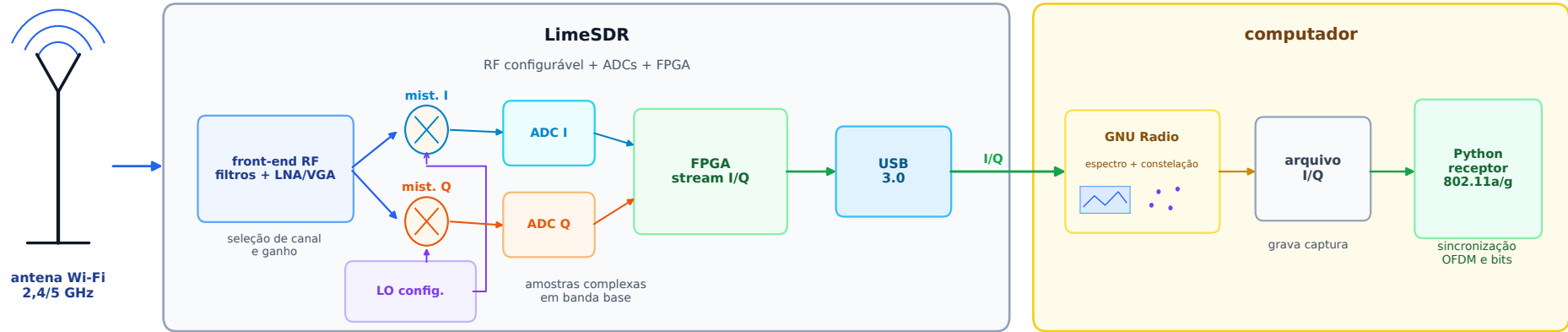
Stefan, *The Direct Sampling SDR Architecture*, obtido de panoradio-sdr.de.

Três arquiteturas de recepção

Arquitetura	Ideia	Vantagem	Risco
Super-heteródino	converte RF para uma ou mais FI antes do ADC	seletividade analógica e bom isolamento	mais filtros, LOs e calibração
Conversão direta	mistura direto para I/Q em banda base	simples e compatível com ADCs moderados	offset DC, vazamento de LO e desequilíbrio I/Q
Amostragem direta	ADC vê RF ou uma FI larga	máxima flexibilidade em software	ADC rápido, faixa dinâmica alta, aliasing exigente

- Muitos SDRs de laboratório usam conversão direta ou FI baixa.
- LimeSDR: RFIC configurável, conversores e FPGA; o host recebe amostras I/Q.
- O código da prática começa depois dessa fronteira: já recebe banda base complexa.

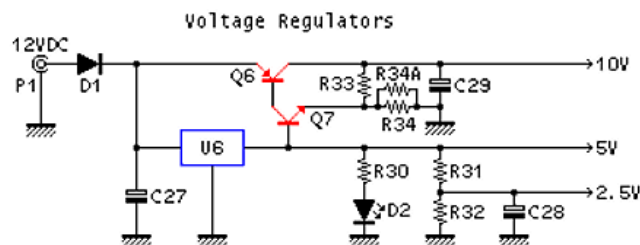
LimeSDR na Prática 5



O LimeSDR não decodifica Wi-Fi sozinho neste módulo:

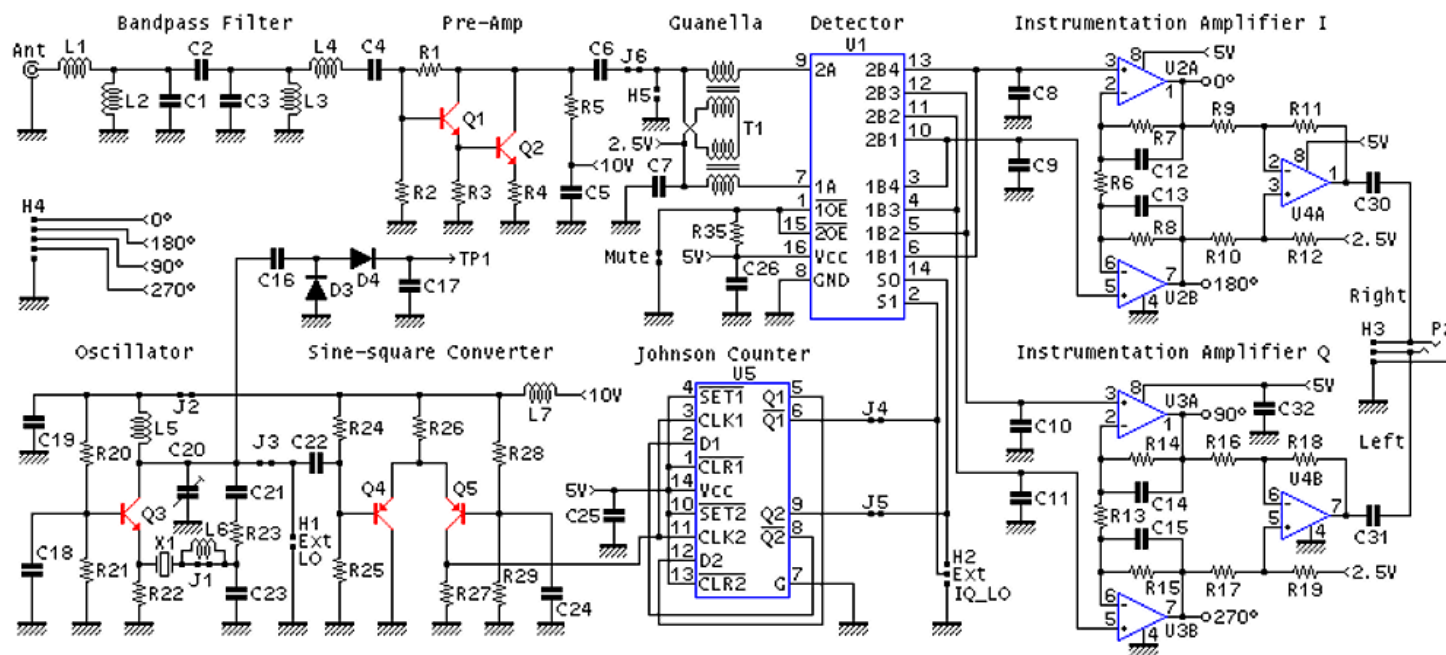
- Ele sintoniza o canal, ajusta ganho e entrega amostras I/Q.
- O GNU Radio ajuda a visualizar espectro, validar captura e salvar arquivo.
- O Python implementa a parte didática do receptor: sincronização, OFDM e bits.
- A captura real inclui imperfeições que a simulação tenta imitar.

Hardware para observar o mundo real, software para abrir a caixa-preta do modem.



SDRZero

Copyright 2006
J. K. De Marco, PY2WM
Edson W. Pereira, PU1JTE, N1VTN



Como ler o circuito do SDRZero

Bloco no esquema	Função	Ponto didático
Filtro de entrada	limita sinais antes do detector	reduz produtos espúrios quando há sinais fortes fora da banda
Pré-amplificador RF	ganho e isolamento em RF	sensibilidade não é só ganho: ruído, impedância e vazamento de LO importam
Guanella	interface entre RF e chaves	transforma a impedância e alimenta o detector de forma balanceada
FST3253 / QSD	chaves amostram a RF em 0°, 90°, 180° e 270°	não é mixer multiplicador convencional; gera I/Q em áudio com baixa perda
Oscilador local	gera a referência senoidal	estabilidade e ruído de fase do LO aparecem diretamente no áudio I/Q
Conversor senóide-quadrada	prepara nível lógico para o contador	o QSD precisa de chaves comandadas por transições digitais limpas
Contador Johnson	produz as quatro fases do LO	a quadratura é feita em hardware antes da placa de som
Amplificadores I/Q	condicionam Right/Left para ADC de áudio	ganho e fase desiguais entre canais reduzem rejeição de imagem

A descrição do SDRZero destaca que ele é um receptor de conversão direta com dois canais em quadratura; o computador faz a demodulação e a escolha de banda lateral em software. Fonte: Descrição do Circuito em wabicafe.com.br.

Do SDRZero ao LimeSDR

Ideia	No SDRZero	No LimeSDR
Sintonia	LO discreto para uma faixa estreita	PLL/RFIC programável por software
Quadratura	contador Johnson e chaves QSD visíveis	misturadores I/Q e calibração dentro do chip
Seleção de canal	filtro de entrada analógico largo + áudio no PC	largura de banda, ganho e taxa configuráveis
Conversão A/D	placa de som estéreo digitaliza I/Q	ADCs rápidos integrados ao SDR
Imperfeições	ganho/fase I/Q, vazamento de LO e ruído ficam evidentes	mesmos defeitos existem, mas são calibrados e transportados em alta taxa
Papel do PC	demodula áudio I/Q e rejeita imagem por DSP	recebe amostras complexas já em banda base e roda receptor Wi-Fi ou outros protocolos

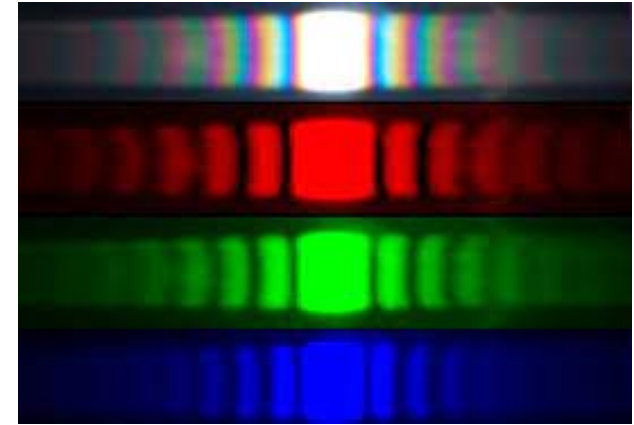
O SDRZero abre a caixa-preta do front-end; o LimeSDR mostra a mesma arquitetura em escala moderna, com integração, controle digital e requisitos de taxa muito maiores.

Por que Wi-Fi usa OFDM?

Wi-Fi em ambiente interno sofre multi-caminho:

- Reflexões em paredes, móveis e pessoas criam cópias atrasadas.
- Em portadora única, um desvanecimento seletivo distorce todos os símbolos.
- Em OFDM, cada subportadora vê um pedaço estreito do canal.
- Algumas subportadoras caem em vales de cancelamento destrutivo.
- Outras ficam em regiões de soma construtiva.
- A equalização vira uma multiplicação complexa por subportadora.
- Interleaving e FEC espalham os bits pelos tons.

Custo: FFT/IFFT, sensibilidade a erro de frequência e maior PAPR (*Peak-to-Average Power Ratio*).



Bill Alsept, Experimento de dupla de fenda de Young.
Foto obtida de physics.stackexchange.com.

Analogia visual: cada frequência cria seu próprio padrão de interferência.

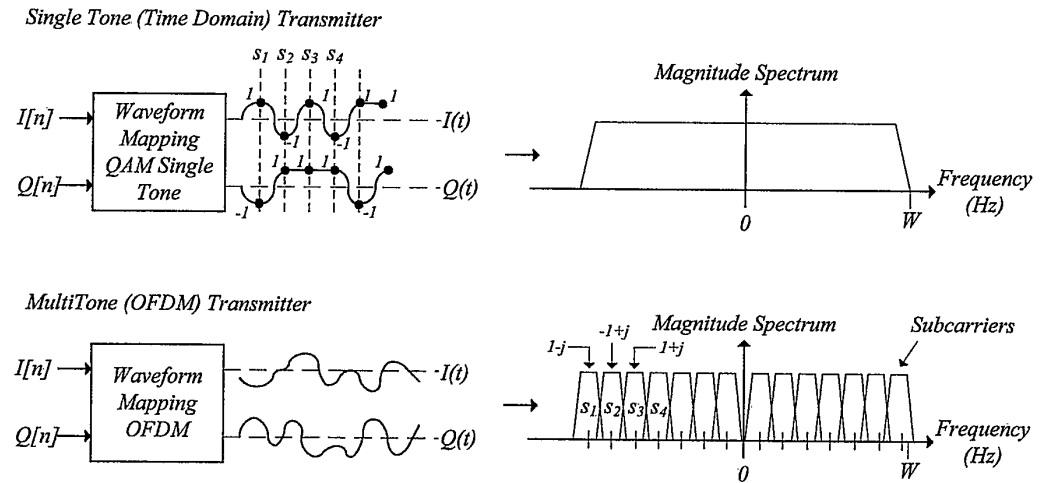
Onde uma cor cancela, outra pode somar. No rádio, multi-caminho cria vales em algumas frequências e preserva outras.

OFDM versus portadora única

A diferença não é a constelação; é o mapeamento para forma de onda.

- Portadora única: símbolos no tempo, depois *pulse shaping*.
- OFDM: blocos de símbolos viram coeficientes de senoides ortogonais.
- No transmissor: IFFT soma as subportadoras.
- No receptor: FFT separa as subportadoras novamente.

Cada símbolo QAM fica difícil de ver no tempo, mas reaparece limpo no domínio da frequência.



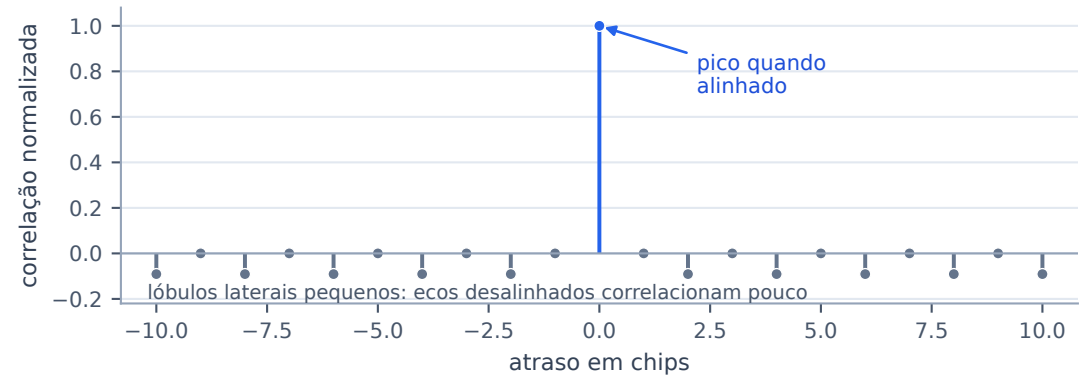
Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-46: OFDM versus Pulse-Shape Mapping.

Antes do OFDM: DSSS em 802.11b

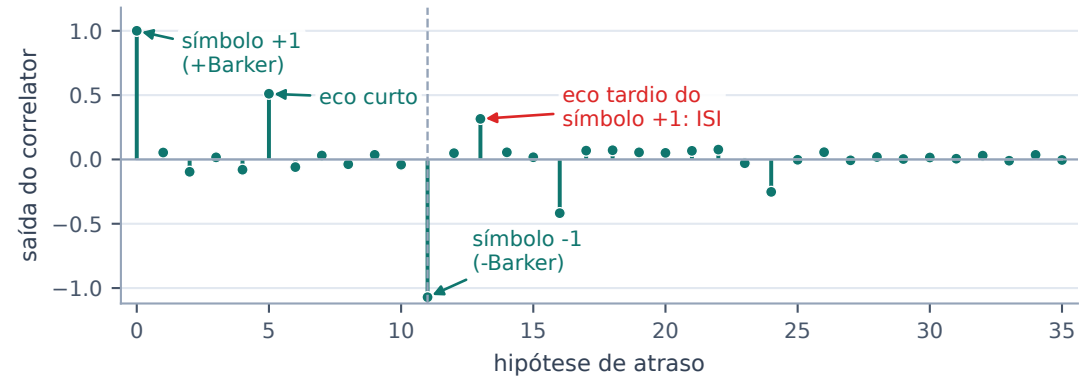
Antes do Wi-Fi OFDM, 802.11b também precisava sobreviver a multi-caminho.

- 1 e 2 Mb/s: DSSS com Barker de 11 chips.
- O receptor correlaciona o sinal recebido com o código esperado.
- Código alinhado: pico forte.
- Ecos resolvíveis: picos atrasados.
- Isso ajuda sincronismo e, em alguns receptores, combinação de caminhos.
- 5,5 e 11 Mb/s: CCK, ainda baseado em chips/correlação, mas mais complexo.
- OFDM venceu: equalização por tom, taxas altas, MIMO e OFDMA.

Autocorrelação do Barker-11



Correlação de dois símbolos com ecos



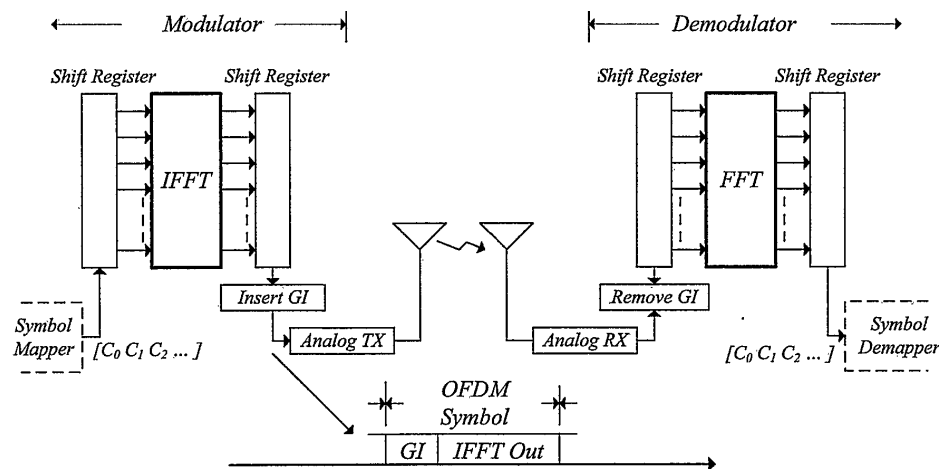
IFFT/FFT: a transformação central

Para um bloco com N subportadoras:

$$x[n] = \sum_{k=0}^{N-1} X[k] e^{j2\pi n \frac{k}{N}}$$

$$X[k] = \left(\frac{1}{N} \right) \sum_{n=0}^{N-1} x[n] e^{-j2\pi n \frac{k}{N}}$$

- $X[k]$: símbolo complexo colocado na subportadora k .
- $x[n]$: amostras no tempo que serão transmitidas.
- Ortogonalidade: a FFT mede a correlação contra cada tom.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-52: Basic Modulation and Demodulation Process in OFDM.

Prefixo cíclico: o truque do OFDM

O prefixo cíclico (CP) cria uma região segura para a janela da FFT.

- O guard interval (GI) é uma cópia do fim do próprio símbolo útil.
- A FFT pode começar um pouco dentro do GI.
- Se “ecos cabem no GI”, existe uma janela de N amostras sem pedaços do símbolo vizinho.
- Dentro dessa janela, cada eco parece um deslocamento circular do mesmo símbolo.

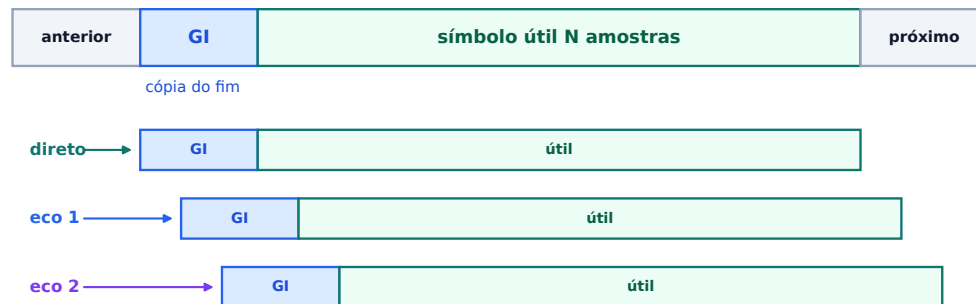
Ideia-chave:

deslocamento no tempo $\rightarrow$ fase na frequência.

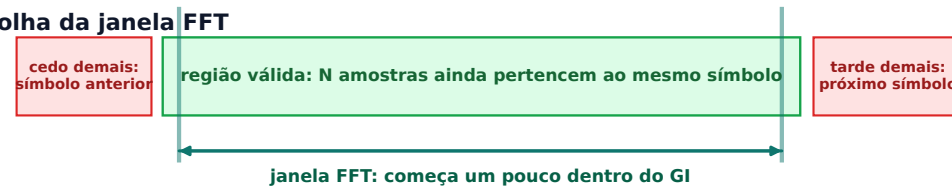
A soma dos ecos vira $H[k]$: $Y[k] = H[k]X[k] + N[k]$.

Em 802.11a/g: útil de $3,2 \mu\text{s}$ + GI de $0,8 \mu\text{s}$ = $4 \mu\text{s}$. Começar cedo reduz o risco de pegar o próximo símbolo.

Símbolo OFDM transmitido



Escolha da janela FFT



Se todos os ecos cabem no GI, existe uma faixa de inícios de FFT sem ISI. Dentro dela, cada eco parece deslocamento circular: na FFT vira fase por subportadora.

Por que o prefixo precisa ser cíclico?

O objetivo do GI é fazer o multipercurso curto parecer uma convolução circular dentro da janela FFT.

No tempo: eco de 1 amostra

$$x = [a, b, c, d], h = [1, \alpha]$$

CP correto: [d | a b c d]

eco atrasado na janela: [d a b c]

$$y = [a + \alpha d, b + \alpha a, c + \alpha b, d + \alpha c]$$

$$y[n] = x[n] + \alpha x[(n - 1) \bmod 4]$$

CP errado: [a | a b c d]

Primeira amostra: $a + \alpha a$.

A circular correta exigia $a + \alpha d$.

Na frequência: multiplicação por tom

Convolução circular:

$$y[n] = \sum_l h[l]x[(n - l) \bmod N]$$

Após a FFT:

$$Y[k] = H[k]X[k]$$
$$H[k] = \sum_l h[l]e^{-j2\pi l \frac{k}{N}}$$

Exemplo numérico

CP correto, só subportadora $k = 2$:

$$x = [1, -1, 1, -1], X = [0, 0, 1, 0]$$

$$h = [1, 0.5]$$

$$H[2] = 1 + 0.5e^{-j\pi} = 0.5$$

$$Y = [0, 0, 0.5, 0]$$

CP errado: [1 | 1 -1 1 -1]

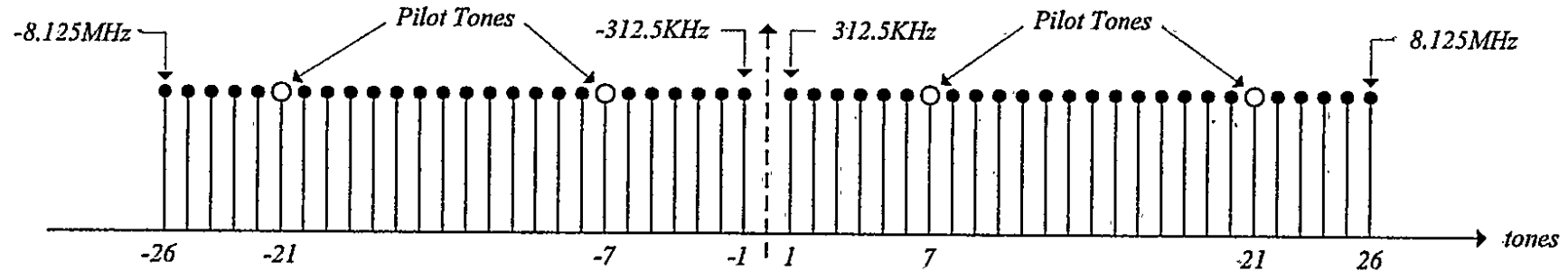
$$y_{\text{err}} = [1.5, -0.5, 0.5, -0.5]$$

$$Y_{\text{err}} = [0.25, 0.25, 0.75, 0.25]$$

A energia vazou para outros bins.

Sem CP correto, a FFT ainda calcula uma DFT; o que se perde é o modelo simples de um ganho independente por subportadora.

OFDM em 802.11a/g



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-15: Distribution of Information Carrying and Pilot Tones.

Parâmetros principais:

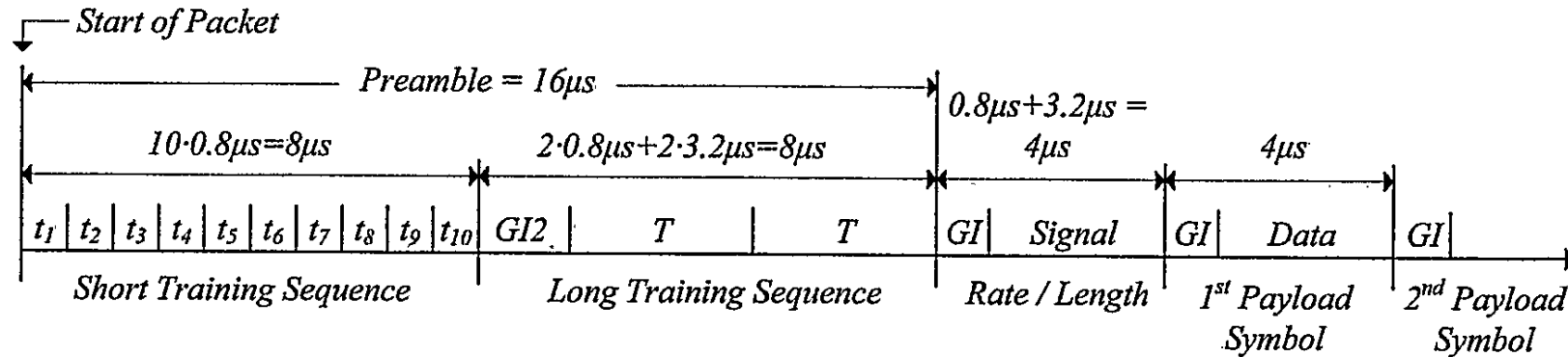
Item	Valor	Função
Taxa de amostragem	20 MS/s	64 amostras em 3,2 μ s
FFT/IFFT	64 pontos	subportadoras espaçadas de 312,5 kHz
Subportadoras úteis	52	48 dados + 4 pilotos
Pilotos	-21, -7, 7, 21	rastrear fase e temporização
Nulas	DC e bordas	evitar DC e criar guarda espectral

Taxas 802.11a/g: bits por símbolo OFDM

Mbps	Modulação	Código	N_{CBPS}	N_{DBPS}
6	BPSK	1/2	48	24
9	BPSK	3/4	48	36
12	QPSK	1/2	96	48
18	QPSK	3/4	96	72
24	16-QAM	1/2	192	96
36	16-QAM	3/4	192	144
48	64-QAM	2/3	288	192
54	64-QAM	3/4	288	216

- A prática usa QPSK 1/2 no testbench: simples para ser fácil de depurar e ainda contém FEC.
- Taxas maiores combinam *puncturing* com LLRs mais elaborados para 16/64-QAM.
 - *Puncturing*: transmitir só parte dos bits codificados para obter taxa 2/3 ou 3/4.
 - LLR (*Log-Likelihood Ratio*): $\log\left(\frac{P(b=1 | y)}{P(b=0 | y)}\right)$; sinal indica o bit mais provável, módulo indica confiança.

Estrutura do pacote 802.11a/g



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-14: WLAN 802.11a Packet Structure.

O preâmbulo é o manual de instruções do receptor:

- STS (*Short Training Sequence*): AGC, detecção de pacote e CFO grosseiro.
- LTS (*Long Training Sequence*): CFO fino, referência de tempo e canal.
- SIGNAL: taxa e comprimento, sempre BPSK 1/2.
- DATA: serviço, PSDU, cauda, padding; modulação depende da taxa.

Sem preâmbulo, a FFT não saberia onde começar nem como desfazer o canal.

Campo SIGNAL: o cabeçalho físico

`create_signal_field()` gera 24 bits antes da codificação:

- RATE: 4 bits que indicam modulação e taxa de código.
- Reserved: 1 bit fixo em zero.
- LENGTH: 12 bits com o tamanho do PSDU em bytes.
- Parity: paridade par dos primeiros 17 bits.
- TAIL: 6 zeros para terminar o codificador no estado zero.

Depois:

- codificação convolucional taxa 1/2 transforma 24 em 48 bits;
- interleaving BPSK reorganiza esses 48 bits;
- mapeamento BPSK coloca um bit por subportadora de dados;
- insere 48 símbolos BPSK e 4 pilotos na IFFT;
- adiciona GI/CP de 16 amostras: SIGNAL dura 4 μ s.

Campo DATA no transmissor

O SIGNAL não é embaralhado; o scrambler começa no campo DATA.

`encode_data_field()` monta a carga útil física:

- SERVICE: 16 zeros; os primeiros bits ajudam a recuperar o estado do scrambler.
- PSDU: quadro MAC mais CRC32.
- TAIL: 6 zeros para forçar estado final conhecido no Viterbi.
- PAD: completa o último símbolo OFDM.

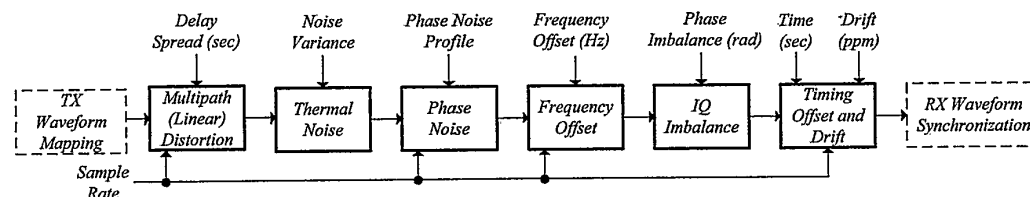
Em seguida, para cada símbolo OFDM de DATA:

- embaralha N_{DBPS} bits de dados;
- codificação e puncturing geram N_{CBPS} bits codificados;
- interleaving reorganiza esses N_{CBPS} bits;
- mapeamento coloca símbolos de constelação nas 48 subportadoras de dados;
- insere 48 símbolos de dados e 4 pilotos na IFFT;
- adiciona GI/CP de 16 amostras: cada símbolo DATA dura 4 μs .

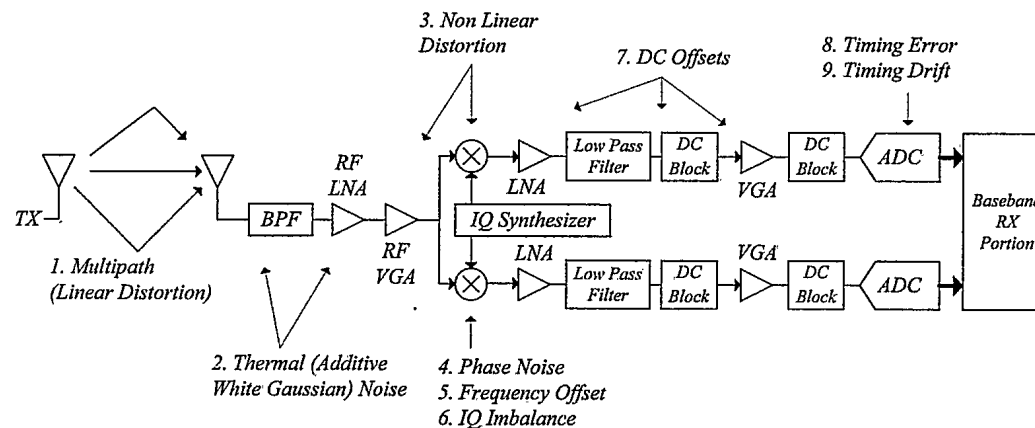
O sinal chega machucado

Defeitos injetados pela prática e discutidos a seguir:

- ThermalNoise: AWGN complexo.
- Multipath: filtro FIR complexo com perfil de atraso.
- Freq_Offset: rotação linear acumulada.
- PhaseNoise: rotação de fase variante no tempo.
- IQ_Imbalance: ganho desigual e erro de quadratura entre I/Q.
- TimingOffset e TimingDrift: erro estático e deriva de amostragem.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 6-16: Overview of the Defect Models and Order of Insertion.



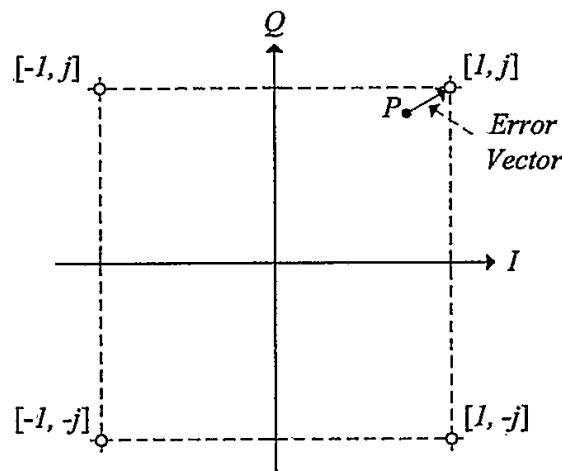
Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 6-3: Distortion, Offset and Noise Sources in a Zero-IF Receiver.

Ruído térmico: o defeito que não se corrige

Ruído térmico adiciona uma componente aleatória: $r = s + w$.

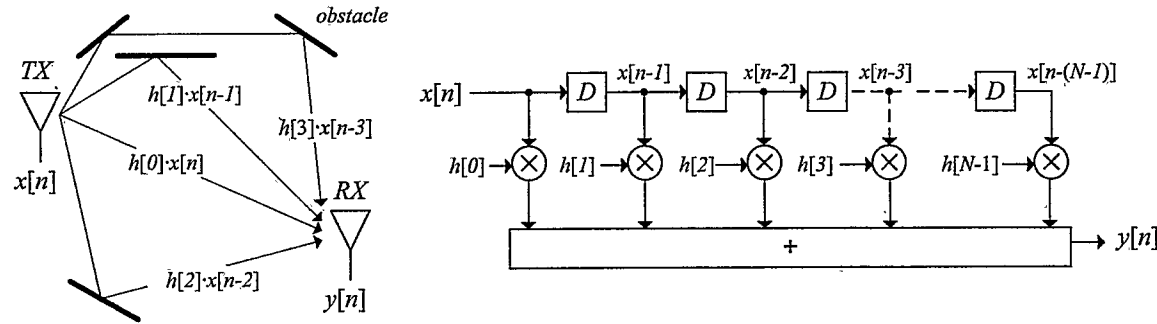
- Abaixo do SNR necessário, não há algoritmo que recupere a informação: limite de Shannon-Hartley.
- O receptor melhora a decisão preservando confiança: *soft bits* e FEC.
- Em OFDM, ruído aparece em todas as subportadoras após a FFT.
- EVM (*Error Vector Magnitude*) mede a distância média entre símbolos corrigidos e pontos ideais; em dB, mais negativo é melhor.

Correção em 802.11: não “remove” AWGN; estima canal, evita degradar mais e usa FEC: BCC/Viterbi em 802.11a/g, LDPC nas normas posteriores.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 6-6: Error Vector in a QPSK Constellation.

Multi-caminho: canal como FIR



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 6-19: Transversal Filter Model of Linear Distortion.

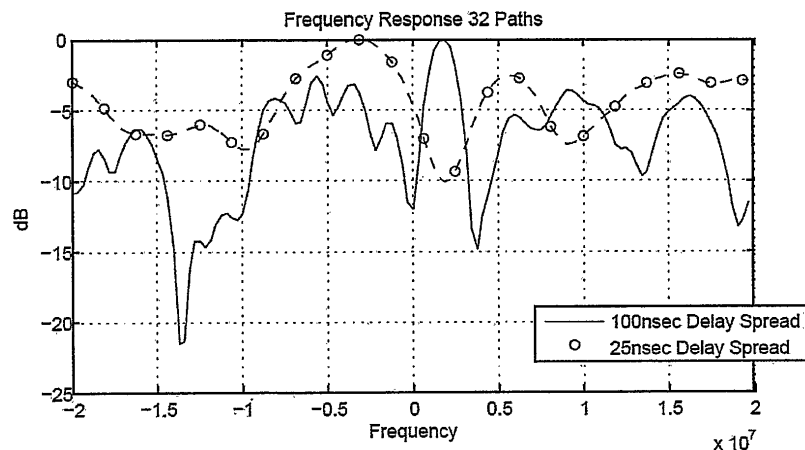
Modelo discreto:

$$y[n] = \sum_l h[l]x[n-l] + w[n]$$

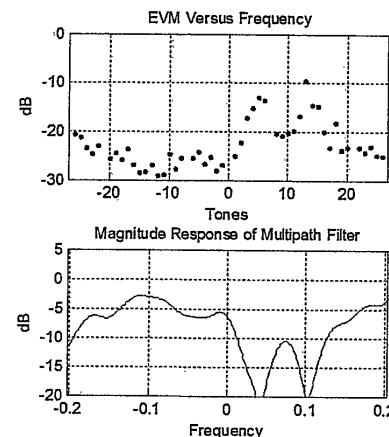
- Cada eco tem atraso, amplitude e fase.
- No tempo: ecos podem causar sobreposição entre símbolos.
- Na frequência: a soma cria vales e picos em $H[k]$.
- OFDM separa o problema em subportadoras estreitas.

Correção em 802.11: GI/CP evita ISI se os ecos cabem nele; LTS estima $H[k]$; equalizador multiplica por $1/H[k]$.

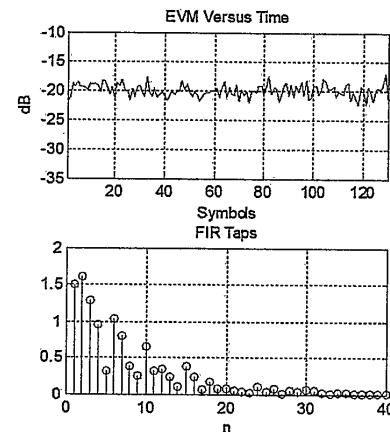
Fading seletivo e EVM por subportadora



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 6-21: Magnitude Response of Channels with 100nsec and 25nsec RMS Delay Spread.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-58: Performance Graphs for SNR = 25dB and Multipath Delay Spread of 100 Nanoseconds.



Leitura prática dos gráficos:

- À esquerda: maior delay spread cria vales mais profundos em $H[k]$.
- Zero-forcing por $1/H[k]$: vale profundo amplifica ruído.
- À direita: EVM vs frequência ruim em tons específicos indica desvanecimento seletivo ou erro de equalização.
- À direita: se EVM vs tempo piorasse ao longo do pacote, sugeriria rastreamento de fase/temporização ruim.
- Interleaving + FEC espalham erros concentrados em poucos tons.

CFO: erro de frequência de portadora

CFO (*Carrier Frequency Offset*) vem de osciladores diferentes:

- Em 2,4 GHz, erro de 20 ppm dá 48 kHz em cada rádio.
- TX e RX em extremos opostos podem produzir quase 100 kHz de erro relativo.
- Em OFDM, CFO desloca subportadoras e quebra ortogonalidade.
- No tempo discreto, aparece como multiplicação por $e^{j2\pi\Delta f \frac{n}{F_s}}$.
- Depois da correção inicial, o resíduo aparece como fase comum por símbolo.

Correção em 802.11:

- STS estima CFO grosseiro com período de 16 amostras.
- LTS estima CFO fino com período de 64 amostras.
- NCO digital multiplica pelo tom oposto para cancelar a rotação.

Ruído de fase e resíduo de CFO

Mesmo depois da correção de frequência, sobra rotação lenta:

- CFO residual: fase acumula símbolo a símbolo.
- Ruído de fase de baixa frequência: rotação comum quase constante dentro de um símbolo.
- Ruído de fase rápido: muda dentro da janela FFT e causa ICI (*Inter-Carrier Interference*), que os pilotos não removem bem.

Correção em 802.11:

- pilotos conhecidos em cada símbolo medem fase comum;
- todos os tons são multiplicados por $e^{-j\theta}$;
- equalizador pode ser atualizado lentamente para rastrear deriva;
- ICI rápido fica como degradação residual de EVM/SNR.

Defeitos de RF fora do fluxo OFDM didático

Eles chegam no vetor I/Q antes da cadeia OFDM:

- **Desequilíbrio I/Q**: ganho/fase diferentes entre I e Q; cria imagem conjugada. A prática modela, mas não corrige.
- **Offset DC**: vazamento de LO e offsets analógicos; aparece em DC. Hardware/driver pode remover antes.
- **Não linearidade**: compressão/clipping em LNA, mixer, VGA/ADC ou PA do TX; AGC evita, mas não desfaz.

Em uma implementação real:

- RFIC/driver pode calibrar I/Q e remover DC antes do script;
- planejamento de ganho evita saturação;
- o receptor didático foca em CFO, janela FFT, equalização, pilotos e FEC.

Erro e deriva de temporização

Temporização tem duas escalas:

- **Aquisição:** escolher a primeira janela FFT correta.
- **Rastreamento:** compensar deriva entre relógios de TX e RX ao longo do pacote.

Efeitos:

- se a janela começa fora da região segura, há ISI;
- se há pequeno deslocamento dentro do prefixo, aparece rampa de fase vs. subportadora;
- se a deriva acumula muito, só corrigir fase pós-FFT deixa de bastar.

Correção em 802.11:

- LTS por correlação define o início da FFT;
- pilotos medem a inclinação de fase por subportadora;
- receptor aplica rampa por tom e atualiza o equalizador;
- em sistemas completos, também ajusta um interpolador.

Defeitos: corrigir pela assinatura

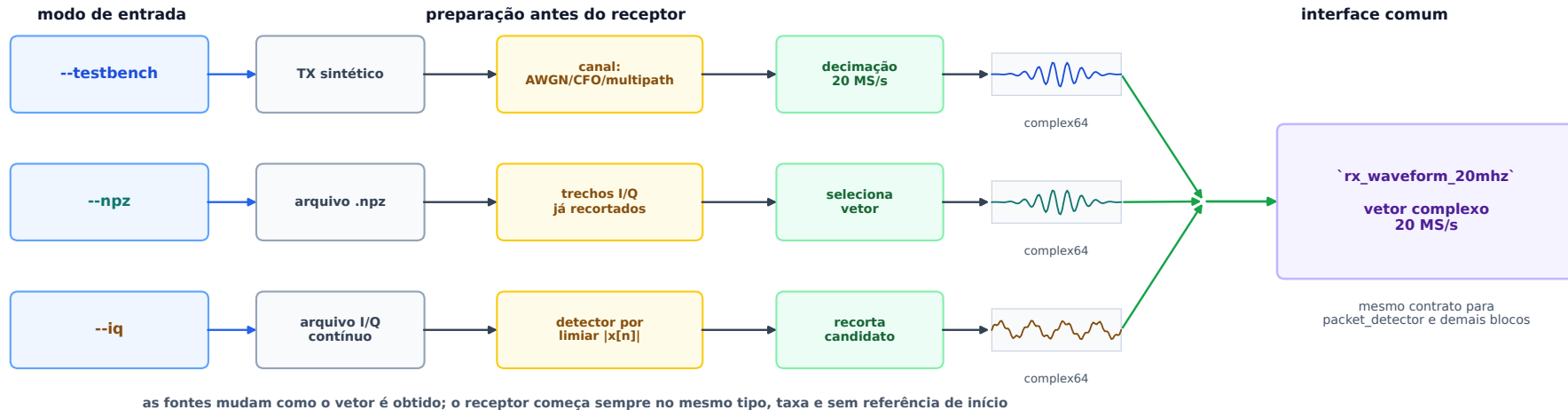
Assinatura observável	Causas que caem nela	Medição	Correção no receptor
Rotação amostra a amostra; ICI se grande	CFO inicial entre TX e RX	autocorrelação STS/LTS	NCO digital antes da FFT
Fase comum por símbolo	CFO residual, fase inicial, ruído de fase lento	fase média dos pilotos	derrotação $e^{-j\theta}$ em todos os tons
Rampa de fase vs. tom	timing offset residual, timing drift/ SFO	inclinação dos pilotos	correção linear por subportadora; atualização lenta
Canal linear estável $H[k]$	multipercurso dentro do CP, filtros, ganho/atraso fixo	LTS conhecida	GI + FFT + equalizador $1/H[k]$
Janela fora da região segura	timing ruim, ecos além do CP	correlação LTS, ISI/EVM	sample_advance; se excede GI, não há correção simples
Ruído/interferência residual	AWGN, quantização, ICI residual	EVM/SNR/LLR	LLR + interleaving + FEC; não remove deterministicamente
Componente em DC	vazamento de LO, offsets de mixer/ amp/ADC	média, tom DC	DC block, subtração de média, tom DC nulo
Imagem conjugada	desequilíbrio I/Q de ganho/fase	imagem, elipse, ganho/fase I/Q	calibração/compensação I/Q fora do fluxo didático

A mesma correção pode servir para causas físicas diferentes quando a assinatura observável é a mesma.

Transformações no código da prática

Como chega	Bloco	Como fica	Por que funciona / por que precisa
amostras I/Q cruas	detector	início marcado	STS repetida faz a autocorrelação subir
pacote com CFO	corrige CFO	CFO inicial reduzido	fase entre repetições estima erro de frequência
tempo sem janela	LTS	janela FFT posicionada	sequência conhecida indica onde o símbolo útil começa
64 amostras no tempo	FFT	64 tons complexos	OFDM transforma eco em ganho por subportadora
tons afetados por $H[k]$	equaliza	tons equalizados	LTS estima $H[k]$; aplica ganho inverso
fase comum residual	pilotos: θ	rotação corrigida	pilotos medem intercepto de fase por símbolo
rampa residual	pilotos: slope	timing rastreado	pilotos medem inclinação de fase vs. tom
64 tons com guardas/pilotos	extraí dados	48 símbolos de dados	802.11 define quais tons carregam carga útil
símbolos QAM/PSK	demapper	LLRs por bit	distância à constelação vira confiança
LLRs interleavados	deinterleaver	LLRs do codificador	desfaz permutação que espalha erros em rajada
LLRs codificados	Viterbi	bits estimados	treliça escolhe caminho de maior métrica
bits embaralhados	descrambler	SERVICE + PSDU + TAIL	LFSR desfaz whitening do transmissor
bits do PSDU	pack + CRC	bytes validados	CRC confirma plausibilidade do quadro

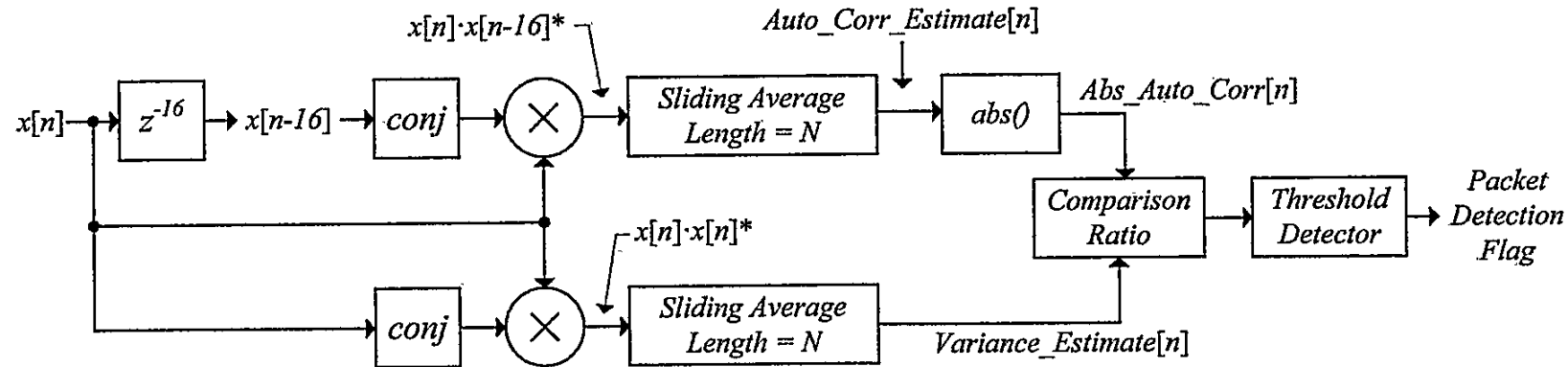
Como o sinal chega ao Python



`rx_waveform_20mhz`: vetor complex64 a 20 MS/s, com ruído, talvez um pacote, CFO, multi-caminho, fase e timing imperfeitos. A referência inicial ainda é nenhuma: a primeira tarefa é descobrir se existe um pacote válido e onde ele começa.

No `--testbench`, o TX/canal rodam a 40 MS/s para aplicar defeitos em uma grade 2x mais fina: taps e timing podem cair em meia amostra do receptor. Depois a decimação entrega o mesmo contrato dos arquivos reais: 20 MS/s.

packet_detector: achar a STS



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-40: 802.11a Packet Detector.

STS no preâmbulo: 10 repetições de 16 amostras (t_1 a t_{10}), total de 8 μ s.

Como chega: amostras I/Q sem referência de início.

Como fica:

- $\text{comparison_ratio}[n] = |R_{16}[n]| / P[n]$;
- $\text{packet_det_flag}[n]$ com histerese;
- $\text{falling_edge_position}$: fim aproximado da STS.

Por que funciona:

- STS repete a cada 16 amostras;
- ruído/interferência não tem essa periodicidade;
- CFO muda apenas fase; multi-caminho preserva a repetição;
- normalização por potência evita depender do ganho absoluto.

Detecção: razão, platô e borda

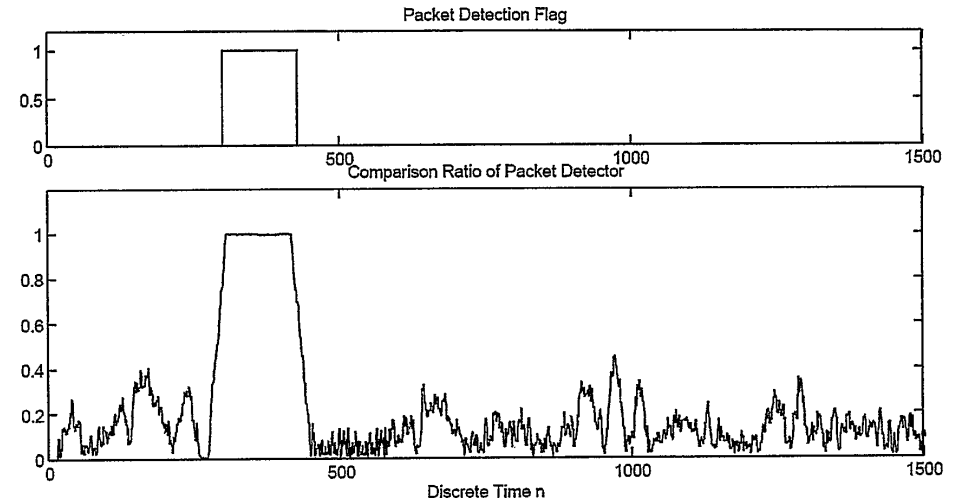
A prática pede um detector robusto, não um pico isolado.

Durante a STS, a periodicidade de 16 amostras dura várias repetições; por isso a razão de autocorrelação forma um platô.

- limiar alto, por exemplo 0,85, liga a detecção;
- limiar baixo, por exemplo 0,65, desliga a detecção;
- a borda de descida marca a transição STS -> LTS.

A borda de subida depende de energia, AGC e limiar; a descida fica mais próxima da mudança estrutural do preâmbulo.

falling_edge_position agenda CFO, LTS e janelas de busca.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-42: Comparison Result as Autocorrelation Estimate/Variance.

detect_frequency_offsets: fase vira Hz

Se uma sequência repete após P amostras:

$$r[n] = a[n]e^{j2\pi\Delta f \frac{n}{F_s}}, \quad a[n] = a[n - P]$$

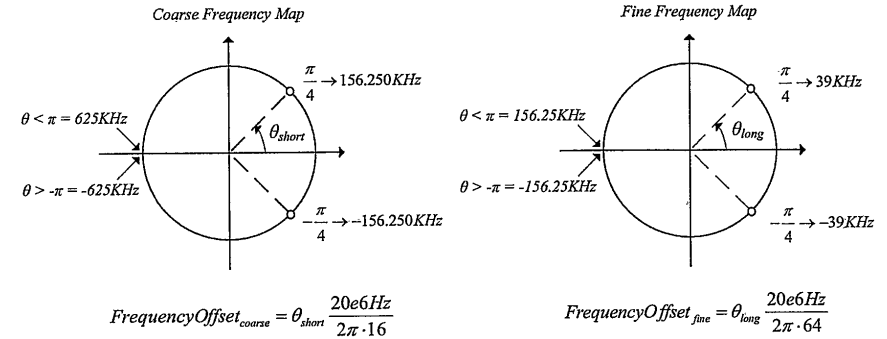
$$r[n]r^*[n - P] = |a[n]|^2 e^{j2\pi\Delta f \frac{P}{F_s}}$$

O produto com conjugado remove a sequência repetida; sobra a rotação acumulada:

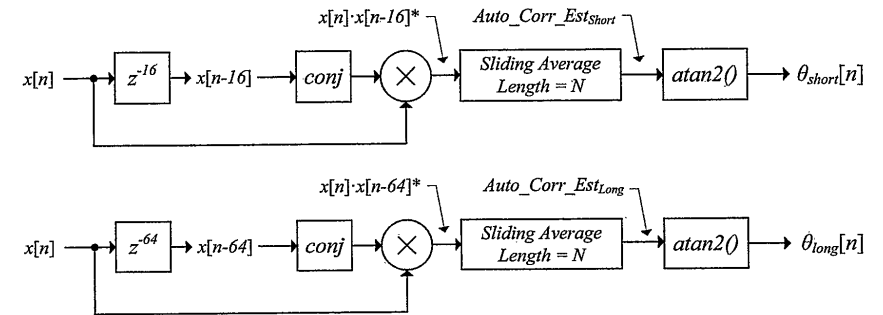
$$\theta = 2\pi\Delta f \frac{P}{F_s}$$

$$\Delta f = \theta \frac{F_s}{2\pi P}$$

- STS: P = 16, faixa de captura maior, precisão menor.
- LTS: P = 64, faixa menor, precisão maior.
- O código mede a fase da autocorrelação em posições agendadas pelo detector.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-43: Correlation Output to Frequency Offset Conversion.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-44: Coarse and Fine Frequency Detectors.

Corrigir CFO: multiplicar por um NCO

Como chega:

- amostras girando no plano complexo por CFO;
- constelação futura giraria símbolo a símbolo;
- FFT sofreria perda de ortogonalidade.

Transformação no código:

```
n = np.arange(len(rx_waveform_20mhz))
nco = np.exp(-1j * 2*np.pi * n * offset / 20e6) # calcula um valor do NCO para cada amostra
rx_waveform_20mhz *= nco # multiplica amostra a amostra, in-place
```

Por que funciona:

- deslocamento em frequência é multiplicação por exponencial complexa;
- multiplicar pela exponencial oposta traz a portadora para zero.

long_symbol_correlator: achar a janela FFT

Como chega: CFO já reduzido; início da FFT ainda incerto.

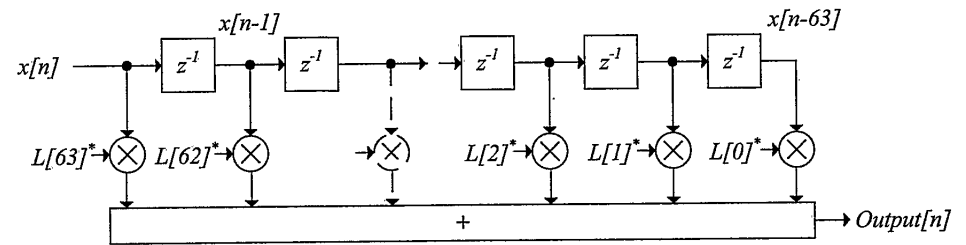
A LTS no preâmbulo é:

GI2 (32 amostras) + T1 (64) + T2 (64).

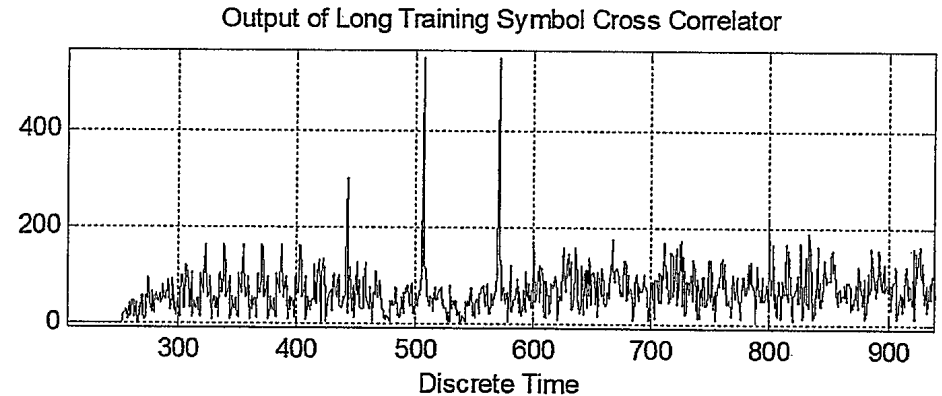
O correlacionador desliza uma LTS local de 64 amostras e gera `output_long[n]`. O pico `lt_peak_position` dá o alinhamento da LTS.

No código:

- pico bruto $\approx$ fim de T1, fronteira T1/T2;
- `p = lt_peak_position - sample_advance`;
- canal: `T1 = rx[p-64:p]`, `T2 = rx[p:p+64]`;
- 1ª FFT pós-preâmbulo: `rx[p+64+16:p+64+16+64]`.

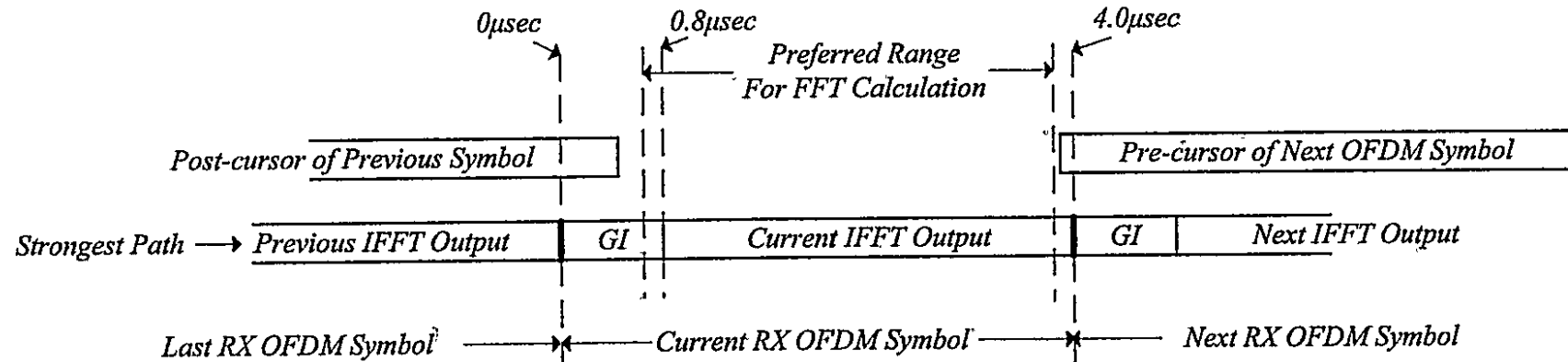


Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-45: Sliding Cross-correlator Implementation for Long Training Sequence.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-46: Output of Long Training Symbol Cross-correlator.

sample_advance: não mirar no limite



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-47: The Preferred Range for the FFT Calculation.

A melhor janela FFT não é necessariamente o pico exato do caminho mais forte.

- Começar cedo demais: pode pegar pós-cursor do símbolo anterior.
- Começar tarde demais: pode pegar pré-cursor do próximo símbolo.
- Começar um pouco dentro do GI: ainda é uma cópia cíclica válida.
- O deslocamento temporal vira rampa de fase, que o equalizador consegue absorver.

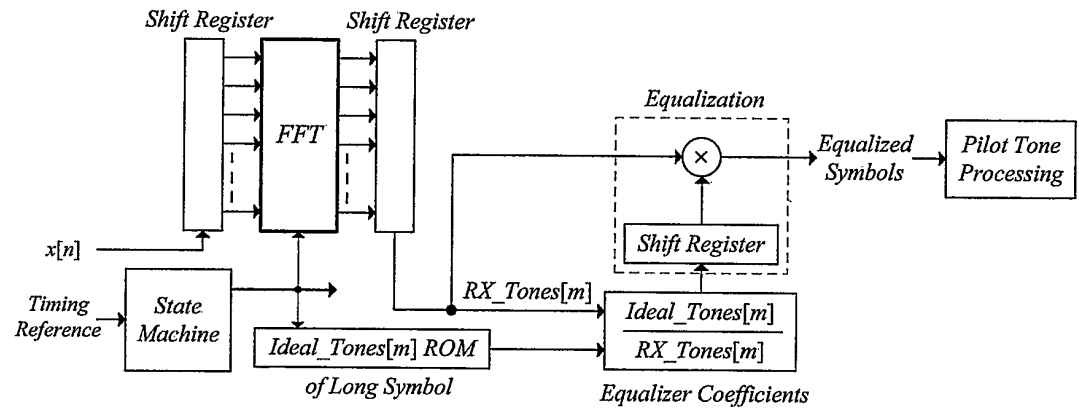
Estimar o canal com a LTS

Na LTS, o transmissor enviou tons conhecidos
 $\text{Ideal_Tones}[k]$.

No receptor:

- extrai duas LTS de 64 amostras;
- faz média para reduzir ruído;
- aplica FFT;
- estima $H[k] = \frac{\text{RX}[k]}{\text{Ideal}[k]}$;
- define coeficientes $E[k] = \frac{1}{H[k]}$.

Como fica: vetor de 64 coeficientes complexos, com tons nulos protegidos contra divisão por zero.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-48: Equalizer Positioning in the OFDM Receiver.

Equalização OFDM: uma multiplicação por tom

Para cada subportadora:

$$Y[k] = H[k]X[k] + N[k]$$

Se $H[k]$ é conhecido:

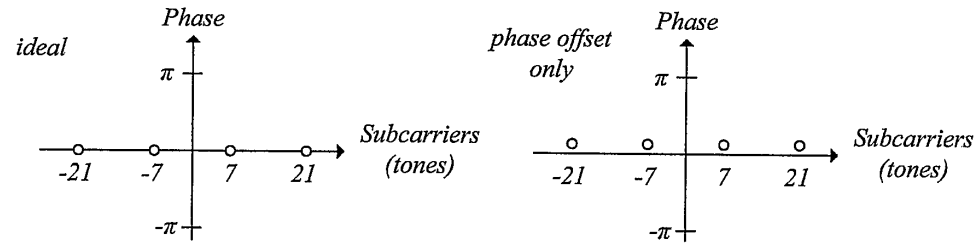
$$\hat{X}[k] = \frac{Y[k]}{H[k]}$$

No código:

- `current_ofdm_symbol`: 64 amostras da janela útil;
- `current_fft_output`: 64 tons complexos;
- `equalized_symbol = current_fft_output * equalizer_coefficients`;
- `DATA_CARRIERS_IDX` extrai os 48 tons de dados.

Por que funciona: em OFDM com GI suficiente, o canal é quase escalar em cada tom.

Pilotos: rastrear fase comum



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-49: Pilot Tone Phases for Varying Scenarios of Phase Offsets.

Como chega:

- símbolo equalizado, mas ainda rotacionado por fase residual;
- pilotos em $[-21, -7, 7, 21]$ têm valores conhecidos.

Transformação:

- remove a polaridade esperada dos pilotos;
- calcula uma média complexa ponderada;
- $\theta = \text{angle}(\text{averaged_pilot})$;
- multiplica todos os tons por $e^{-j\theta}$.

Resultado: constelação recentrada angularmente para aquele símbolo OFDM.

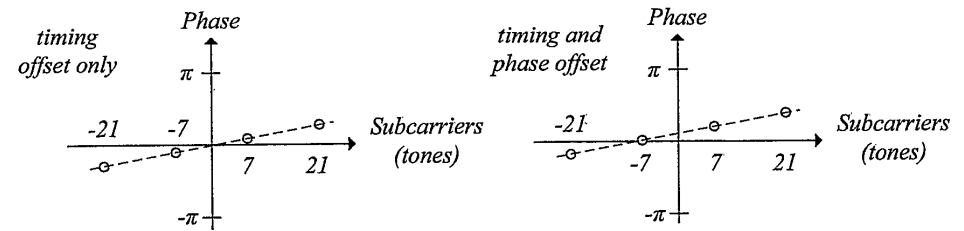
Pilotos: rastrear rampa de fase

Um atraso no tempo vira fase linear na frequência:

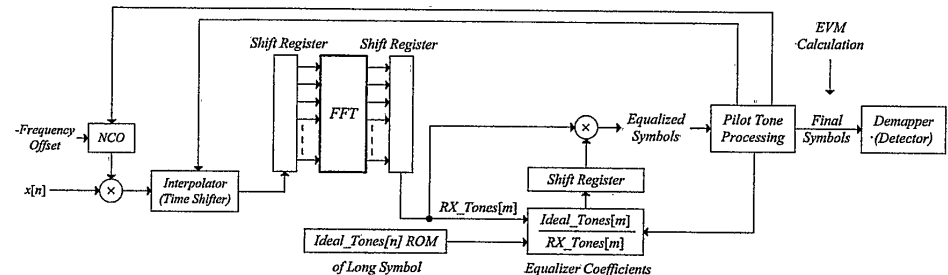
$$x(t - t_0) \leftrightarrow X(f)e^{-j2\pi ft_0}$$

No código:

- `mede angle(pilots);`
- divide pela posição dos pilotos;
- combina com pesos `mrc_coef`;
- suaviza em `average_slope_filter`;
- corrige cada tom por uma rampa de fase.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-50: Pilot Phases for Negative Timing Offset ($t_o < 0$).



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-51: Pilot Processing Positioning in OFDM Receiver.

MRC: confiar mais no piloto forte

Os quatro pilotos não têm a mesma qualidade quando há fading seletivo.

- Se $|H[k]|$ é pequeno, aquele piloto tem SNR pior depois da equalização.
- Média simples dá o mesmo peso para uma medida confiável e uma quase apagada.
- MRC (*Maximum Ratio Combining*) pondera pela força estimada do canal.

No código:

- `pilot_strength = abs(channel_estimate[PILOT_CARRIERS_IDX]);`
- `mrc_coef = pilot_strength / sum(pilot_strength);`
- os mesmos pesos estimam fase comum e inclinação de timing.

É uma melhoria de implementação: a norma define pilotos, não obriga o algoritmo exato de rastreamento.

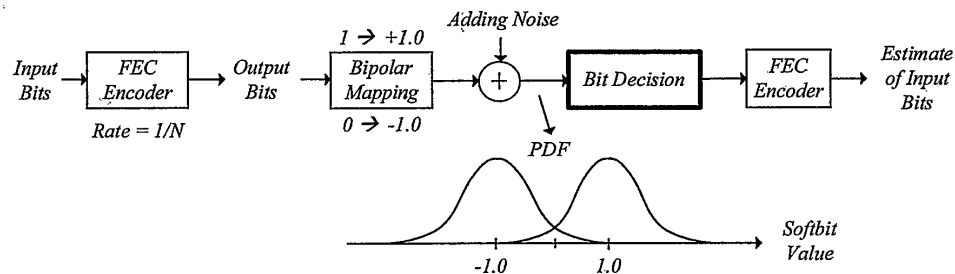
Loop principal: um símbolo por vez

Para cada símbolo OFDM:

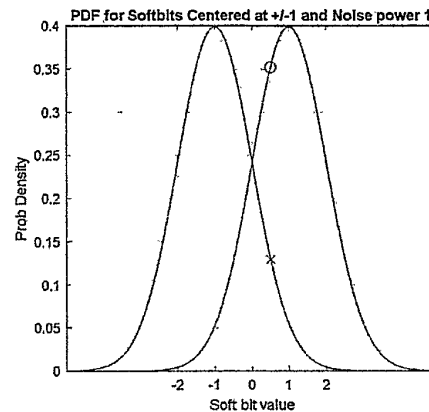
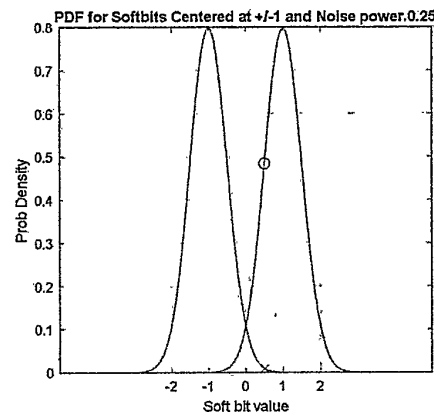
1. Calcula start e stop a partir de `lt_peak_position`, 64 amostras úteis e GI.
2. Extrai 64 amostras da janela FFT.
3. Aplica FFT e equalizador.
4. Usa pilotos para corrigir fase comum.
5. Usa pilotos para corrigir rampa de temporização.
6. Atualiza lentamente coeficientes do equalizador.
7. Guarda só as 48 subportadoras de dados.

Depois deste ponto, o sinal deixou de ser uma forma de onda: agora é uma sequência de símbolos de constelação.

demapper_ofdm: símbolo vira confiança de bit



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-68: Hard or Soft Bits Decisions.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-69: Softbits with Varying Noise Powers.

Hard decision joga fora informação:

- +0.05 e +5.0 virariam o mesmo bit 1.
- Viterbi soft precisa saber confiança, não só sinal.

Na prática:

- BPSK: LLR aproximado = parte real.
- QPSK: LLRs aproximados = parte real e parte imaginária intercaladas.

LLR simplificado usado na Prática 5

Para BPSK/QPSK equalizado e ruído aproximadamente constante:

- sinal positivo favorece bit 1;
- sinal negativo favorece bit 0;
- magnitude maior = maior confiança.

BPSK

```
soft_bits = np.real(symbols_iq) # aplica a todos os símbolos e devolve um vetor real
```

QPSK

```
soft_bits[0::2] = np.real(symbols_iq) # preenche posições pares: 0, 2, 4, ...
```

```
soft_bits[1::2] = np.imag(symbols_iq) # preenche posições ímpares: 1, 3, 5, ...
```

Para 16-QAM e 64-QAM, cada bit precisa comparar grupos de níveis possíveis; a aproximação por eixo não basta.

Por que código convolucional + interleaving?

BCC (*Binary Convolutional Coding*) é um código convolucional binário: cada bit afeta uma sequência curta de bits codificados, que depois é decodificada por Viterbi.

Reed-Solomon no Módulo 4

- Código de bloco sobre símbolos, tipicamente bytes.
- Corrige rajadas curtas naturalmente: vários bits errados podem contar como um símbolo errado.
- Muito útil quando a camada física entrega decisões mais duras por byte/bloco.
- Interleaving ainda ajuda se a rajada excede a capacidade de correção do bloco.

802.11a/g OFDM

- O demapper gera LLRs por bit.
- BCC + Viterbi usa essa confiança bit a bit.
- Fading seletivo cria subportadoras ruins; o interleaver espalha esses erros antes do Viterbi.
- Baixa latência e hardware previsível para pacotes curtos.

A evolução do Wi-Fi também não foi para RS: 802.11n/ac/ax/be adicionam LDPC, que preserva a lógica de usar informação soft e escala melhor para taxas altas.

deinterleaving_pattern: desfazer a permutação

Interleaving no transmissor faz duas coisas:

- evita que bits consecutivos do codificador caiam em subportadoras adjacentes;
- em QAM alta, espalha bits entre posições de significância diferente.

No receptor:

- `soft_bits` chega na ordem em que os símbolos foram demapeados;
- `deinterleaving_pattern` reordena LLRs para a ordem do codificador;
- o Viterbi volta a enxergar a treliça na ordem correta.

Sem deinterleaving, o Viterbi receberia ramos embaralhados: a métrica seria calculada contra o tempo errado.

descramble: desfazer o branqueamento

Scrambling não é criptografia.

- Evita longas sequências de 0 ou 1.
- Ajuda a manter espectro e estatísticas mais estáveis.
- Usa LFSR com polinômio $S(x) = x^7 + x^4 + 1$.
- XOR é sua própria inversa: aplicar a mesma sequência desfaz a operação.

Em 802.11:

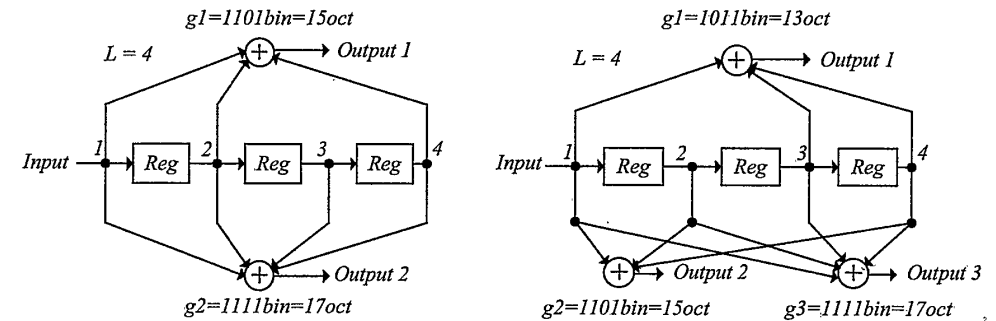
- SIGNAL não passa pelo scrambler – o scrambler começa a ser sincronizado no DATA;
- os 7 LSBs do campo SERVICE (primeiro campo de DATA) são zero antes do scrambling;
- o receptor usa esses bits para estimar o estado inicial;
- depois desembaralha SERVICE + PSDU + TAIL + PAD.

Código convolucional (BCC) em 802.11

BCC (*Binary Convolutional Coding*) é o código convolucional binário usado pelo 802.11a/g clássico. A prática usa:

- taxa mãe: $1/2$;
- constraint length: $L = 7$;
- memória: $m = L - 1 = 6$ bits, portanto 64 estados;
- geradores: 133 e 171 em octal;
- cada bit de entrada produz dois bits codificados.

Taxas $3/4$ e $2/3$ são obtidas por *puncturing*: alguns bits codificados deixam de ser transmitidos.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-76: Rate 1/2 and Rate 1/3 Convolutional Encoders.

A figura ilustra encoders menores do livro, com $L = 4$; o Wi-Fi usa $L = 7$.

Como os geradores BCC são escolhidos?

Os geradores definem quais taps do registrador entram em cada XOR de saída. Para taxa mãe $1/r$, há r polinômios geradores. Com memória $L - 1$, o espaço bruto tem até $(2^L - 1)^r$ combinações; para cada candidata, calcula-se a distância livre d_{free} e o espectro de distâncias, depois valida-se por simulação.

O que se calcula para cada candidato:

- evitar códigos catastróficos: uma entrada com quantidade de 1s tendendo a infinito não pode gerar uma saída com quantidade finita de 1s;
- distância livre d_{free} : menor distância de Hamming entre dois caminhos codificados distintos;
- espectro de distâncias: quantos caminhos errados existem perto de d_{free} .

O que isso garante:

- maior d_{free} separa melhor o caminho correto dos caminhos errados;
- melhora a probabilidade de erro do Viterbi, especialmente com métricas soft;
- não dá uma regra simples “corrige até t erros” como Reed-Solomon;
- erros próximos, posições e confiabilidade das amostras importam para a decisão.

Escolher L é outro compromisso: maior L pode melhorar o código, mas o Viterbi cresce como 2^{L-1} estados.

O par 133/171 em octal já era um código convolucional clássico de taxa $1/2$ e $L = 7$; o IEEE 802.11 adotou essa escolha conhecida.

Exemplo trabalhado: encoder pequeno

Para entender Viterbi, use um encoder menor: taxa $1/2$, $L = 3$, geradores 111 e 101.

Estado inicial 00; entrada 1 0 1 1 seguida de cauda 0 0 para terminar em estado conhecido:

bit	estado antes	saída	estado depois
1	00	$1 \oplus 0 \oplus 0 = 1$, $1 \oplus 0 = 1 \rightarrow 11$	10
0	10	$0 \oplus 1 \oplus 0 = 1$, $0 \oplus 0 = 0 \rightarrow 10$	01
1	01	$1 \oplus 0 \oplus 1 = 0$, $1 \oplus 1 = 0 \rightarrow 00$	10
1	10	$1 \oplus 1 \oplus 0 = 0$, $1 \oplus 0 = 1 \rightarrow 01$	11
0 (tail)	11	$0 \oplus 1 \oplus 1 = 0$, $0 \oplus 1 = 1 \rightarrow 01$	01
0 (tail)	01	$0 \oplus 0 \oplus 1 = 1$, $0 \oplus 1 = 1 \rightarrow 11$	00

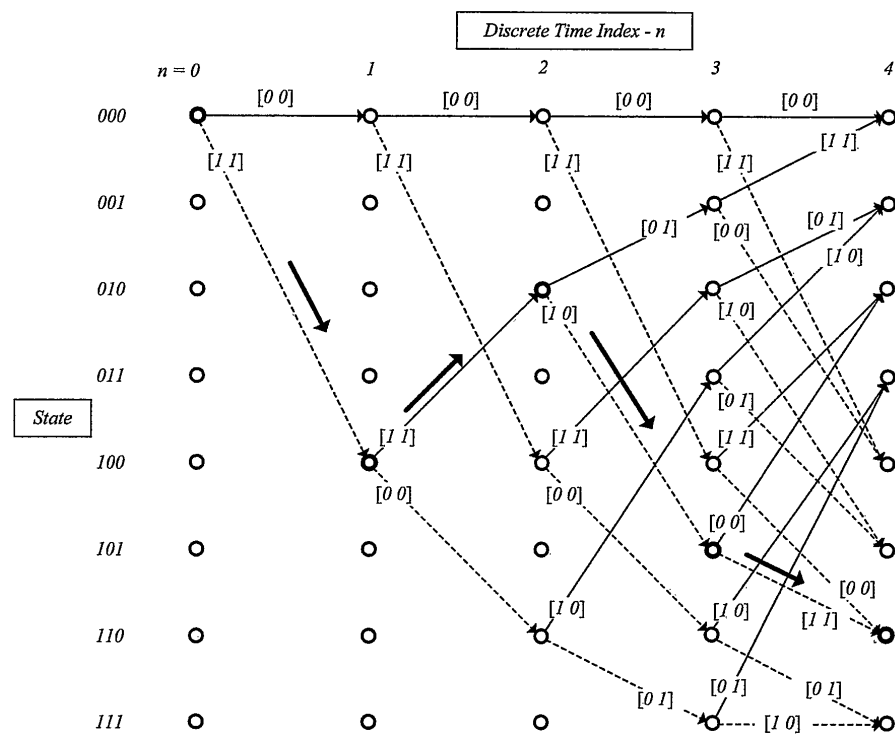
Saída codificada: 11 10 00 01 01 11.

O decoder conhece a máquina de estados. Ele procura qual caminho pela máquina explica melhor a sequência recebida.

Treliça: memória vira grafo no tempo

A treliça organiza todas as hipóteses válidas.

- Cada coluna é um instante de entrada.
- Cada linha é um estado do registrador.
- Cada nó tem duas saídas possíveis: bit 0 ou bit 1.
- O caminho completo codifica a mensagem completa.

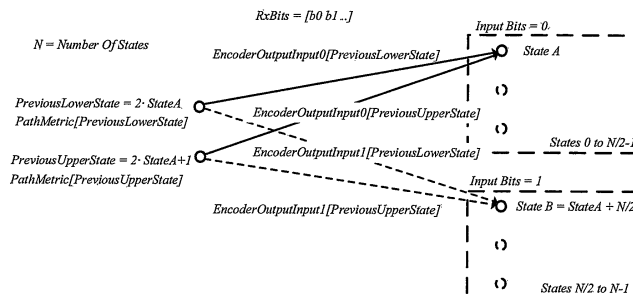


Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-79: Trellis Diagram.

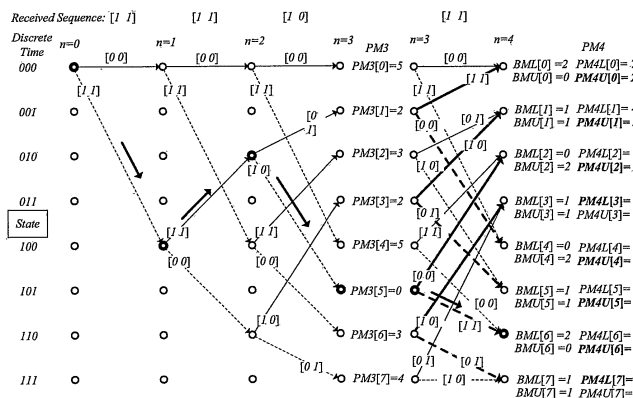
Viterbi: métrica, sobrevivente, traceback

Algoritmo:

1. Inicialize métricas: estado zero bom, demais ruins.
2. Para cada par de bits recebidos, compute métricas de ramo.
3. Para cada estado destino, compare os dois caminhos que chegam nele.
4. Guarde só o melhor caminho: o sobrevivente.
5. No fim, faça traceback para recuperar os bits.



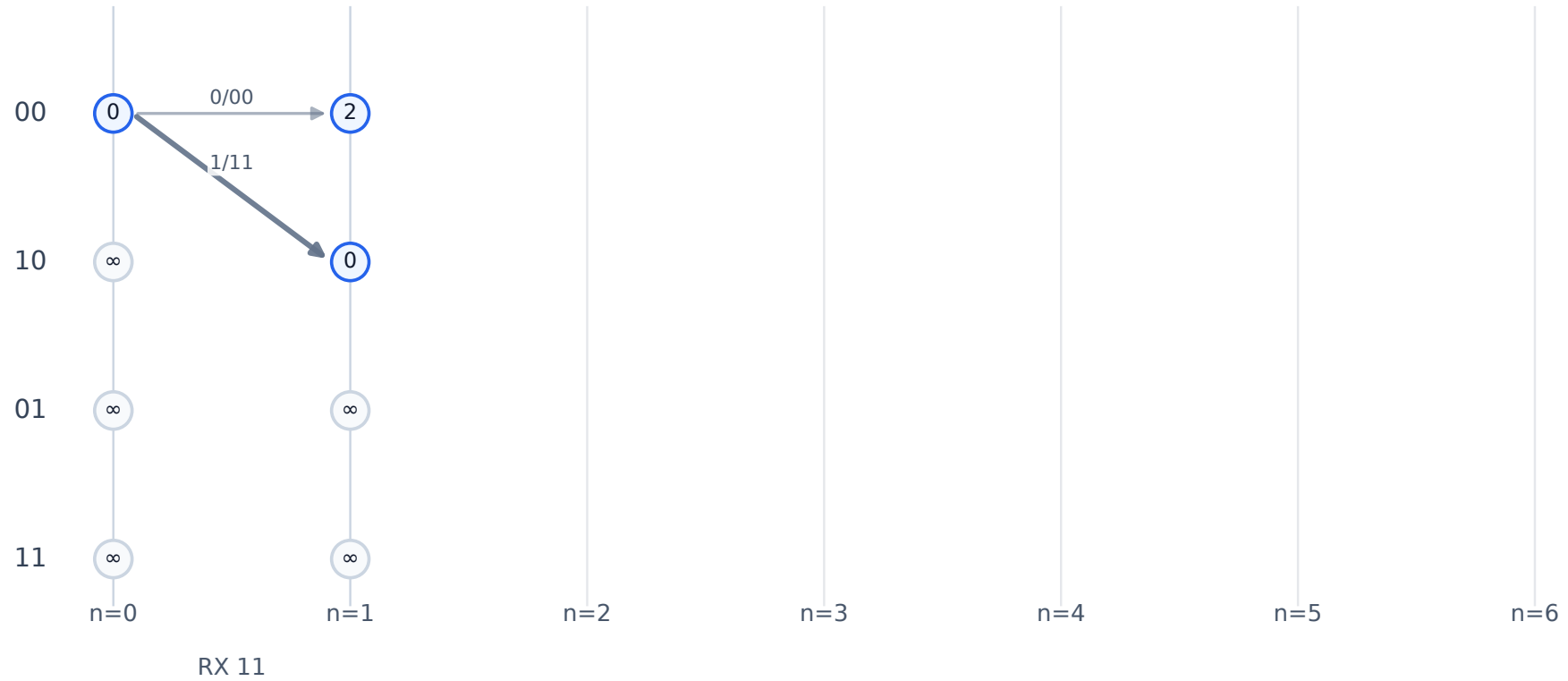
Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-83: Viterbi Iteration.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-82: Path and Branch Metrics.

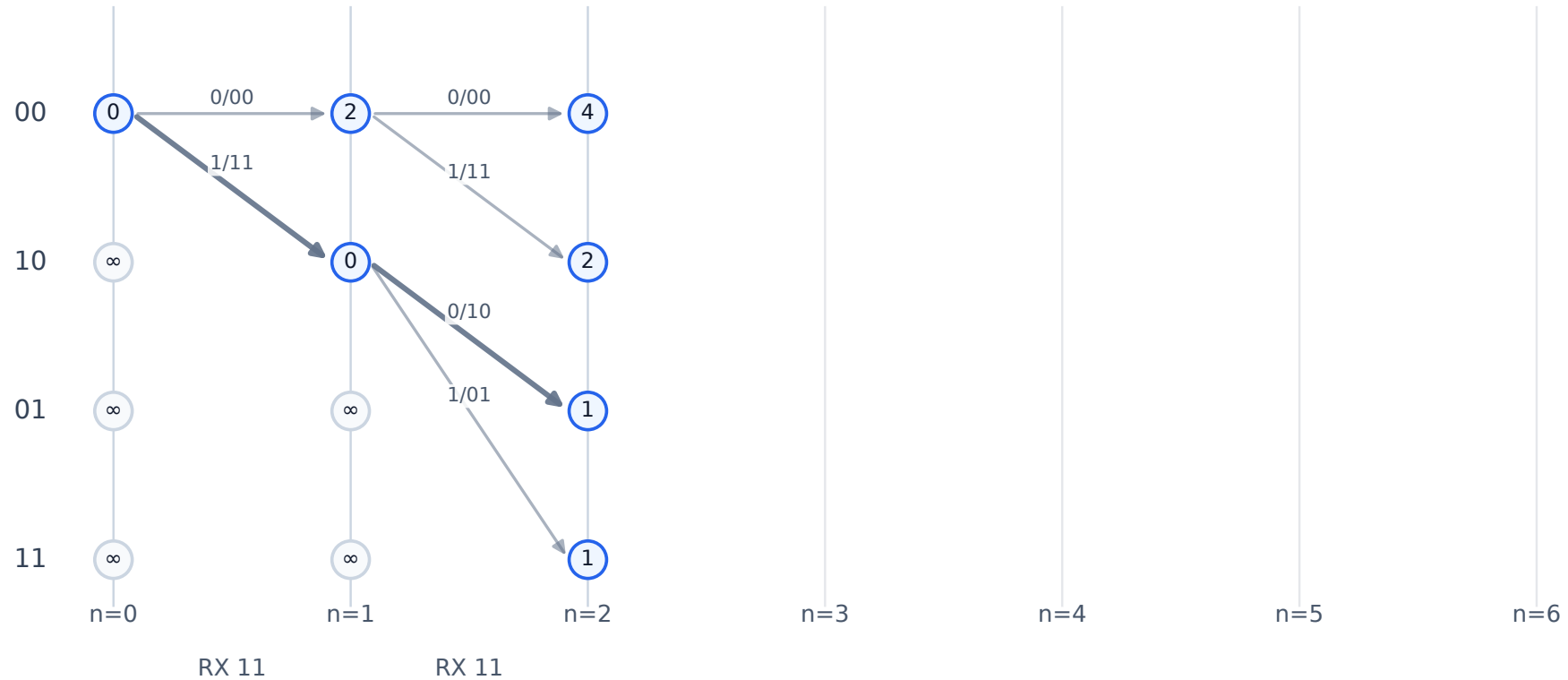
Passo 1: métricas partem do estado 00

Cada nó mostra a melhor métrica acumulada até aquele estado.



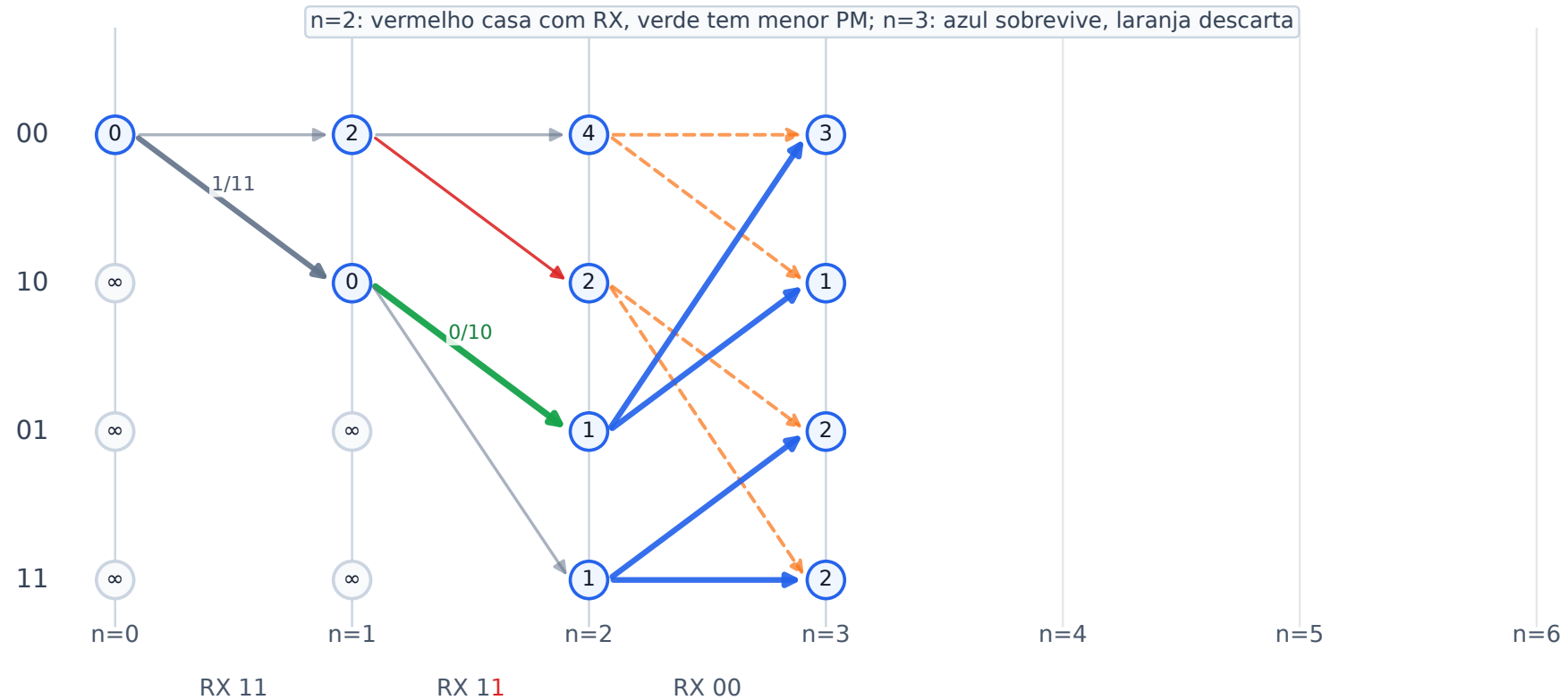
Passo 2: atualiza métricas sem decidir a mensagem

Com apenas dois pares, ainda só propagamos custos pela treliça.



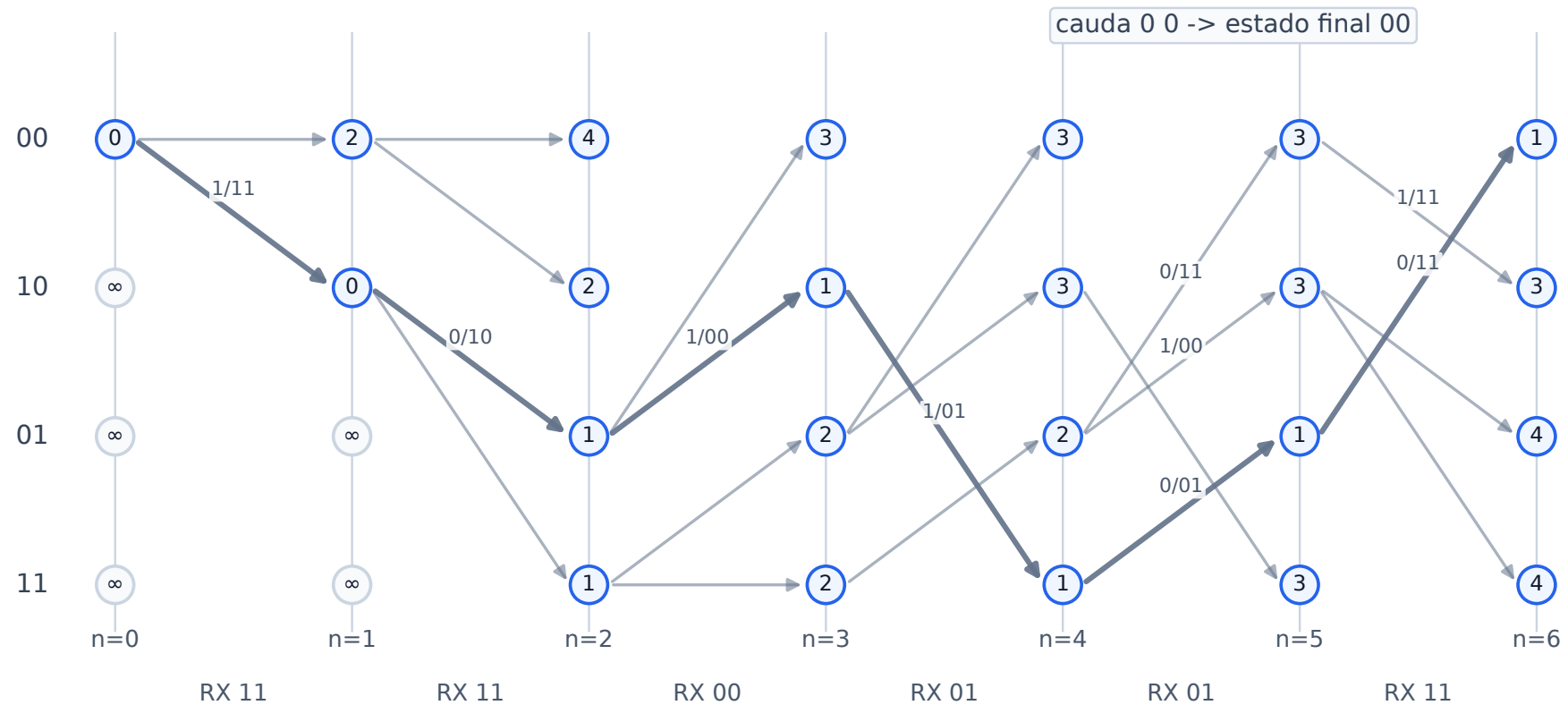
Passo 3: pela primeira vez há descarte por estado

Quando dois caminhos chegam ao mesmo estado, só o de menor métrica sobrevive.



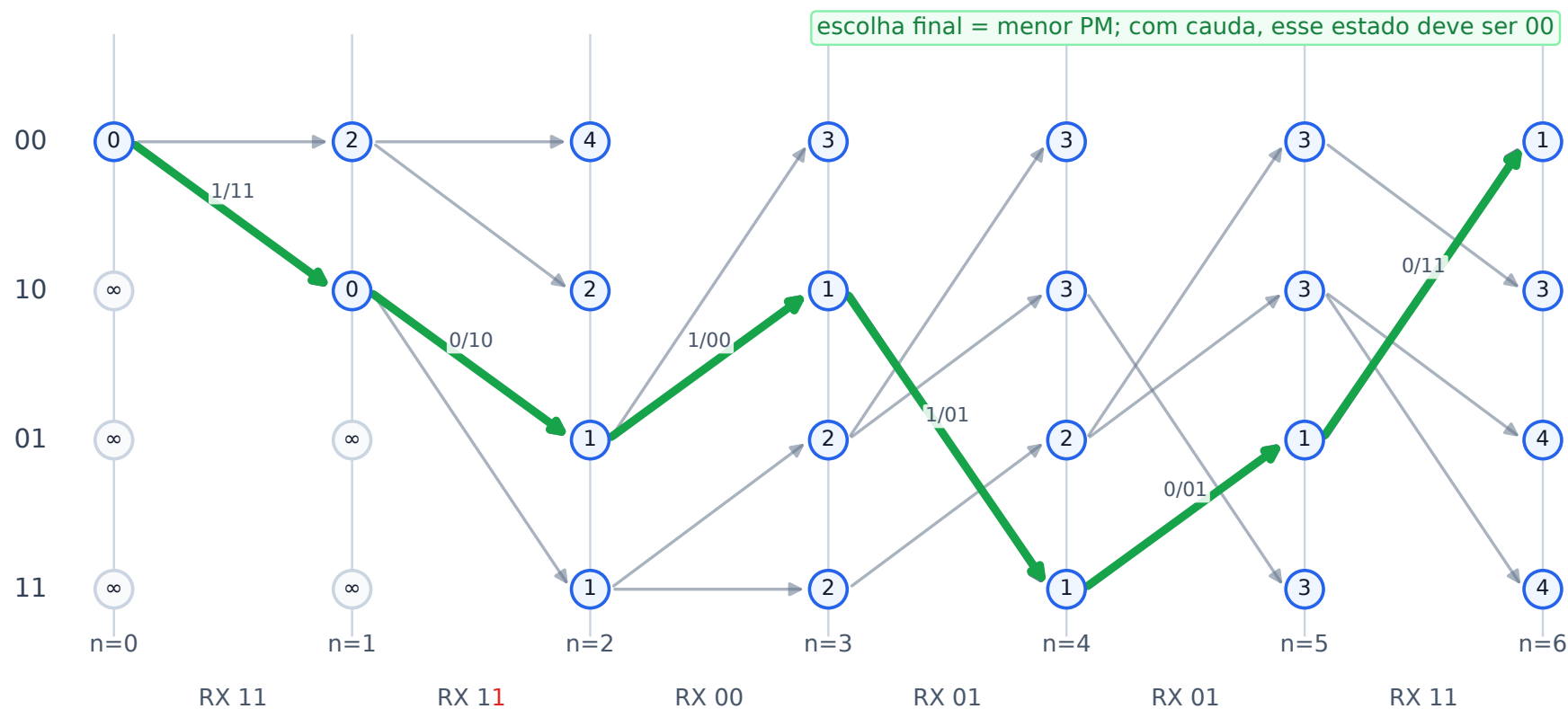
Passo 4: processa toda a sequência, incluindo a cauda

A cauda 0 0 torna conhecido o estado final do encoder pequeno.



O caminho sobrevivente revela a sequência de entrada mais provável.

O caminho sobrevivente revela a sequência de entrada mais provável.



Viterbi soft-decision

Diferença principal:

- Hard-decision: ramo custa a quantidade de bits diferentes.
- Soft-decision: ramo pontua correlação entre LLRs recebidos e bits esperados em forma bipolar.

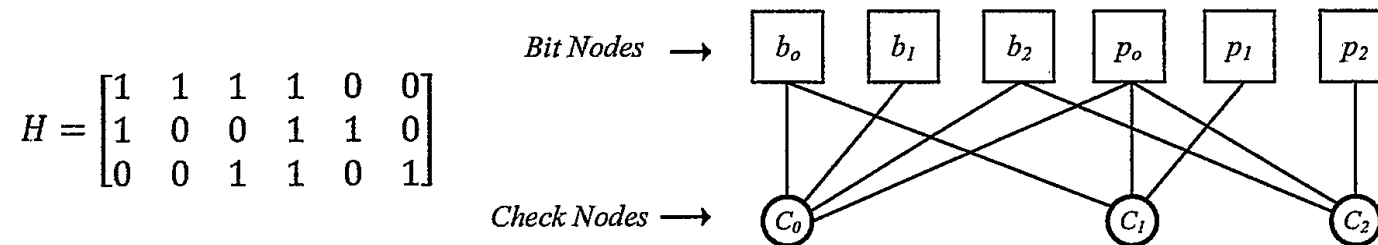
Para mapeamento 802.11, 0 $\rightarrow$ -1, 1 $\rightarrow$ +1:

$$BM = \sum_i r_i b_i$$

- r_i : LLR ou softbit recebido.
- b_i : saída esperada do encoder em {-1, +1}.
- maximizar BM em vez de minimizar custo: maior métrica = melhor compatibilidade.

Soft-decision costuma dar cerca de 2 dB sobre hard-decision porque preserva confiança.

LDPC: matriz esparsa e grafo de Tanner



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-88: Parity Check Matrix and Tanner Graph.

LDPC é um código de bloco definido por uma matriz H .

- A palavra-código junta bits de dados e paridade: $x = [b_0 b_1 b_2 p_0 p_1 p_2]$.
- Palavra válida satisfaz $Hx^T = 0$ em GF(2): cada linha de H impõe uma soma XOR que deve dar zero.
- “Low density” significa poucas conexões por linha/coluna.
- O grafo de Tanner alterna nós de bit e nós de paridade.
- Decodificação iterativa troca crenças entre os nós: aqui, “crença” é um LLR sobre um bit.

802.11n/ac/ax/be usam LDPC como alternativa moderna ao código convolucional binário; 802.11a/g clássico usa BCC com decodificação por Viterbi.

Exemplo trabalhado: encoder LDPC pequeno

Usando a matriz do slide anterior, a palavra-código é $x = [b_0 b_1 b_2 p_0 p_1 p_2]$.

Cada check impõe uma soma XOR igual a zero:

check	equação	para $b = [1, 0, 1]$
C_0	$b_0 \oplus b_1 \oplus b_2 \oplus p_0 = 0$	$p_0 = 1 \oplus 0 \oplus 1 = 0$
C_1	$b_0 \oplus p_0 \oplus p_1 = 0$	$p_1 = 1 \oplus 0 = 1$
C_2	$b_2 \oplus p_0 \oplus p_2 = 0$	$p_2 = 1 \oplus 0 = 1$

Resultado: $x = [1, 0, 1, 0, 1, 1]$.

Verificação: $Hx^T = [0, 0, 0]$, portanto a palavra é válida.

Este exemplo é didático e pequeno; matrizes LDPC reais de Wi-Fi são muito maiores, mas obedecem à mesma regra $Hx^T = 0$.

Exemplo trabalhado: decodificação por síndrome

Suponha que o canal inverteu o bit b_2 :

$$x = [1, 0, 1, 0, 1, 1] \rightarrow y = [1, 0, \textcolor{red}{0}, 0, 1, 1]$$

Calculamos a síndrome $s = Hy^T$:

check	soma XOR recebida	síndrome
C_0	$1 \oplus 0 \oplus 0 \oplus 0$	1
C_1	$1 \oplus 0 \oplus 1$	0
C_2	$0 \oplus 0 \oplus 1$	1

Falharam C_0 e C_2 . A coluna de b_2 em H é $[1, 0, 1]^T$, igual à síndrome.

Corrigimos b_2 : $[1, 0, 0, 0, 1, 1] \rightarrow [1, 0, 1, 0, 1, 1]$.

Com múltiplos erros, seria preciso buscar quais colunas de H , combinadas por XOR, produzem a síndrome; isso não escala. Veremos a seguir o min-sum, que usa os mesmos checks no grafo, mas propaga LLRs.

Algoritmo min-sum

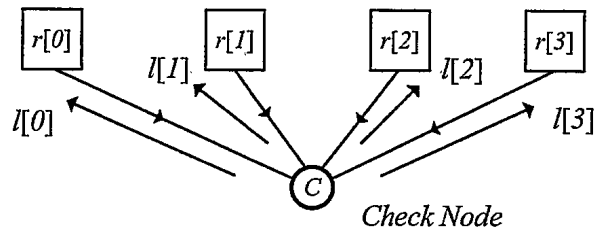
Uma iteração LDPC min-sum:

1. Bits enviam crenças atuais para checks conectados.
2. Cada check é uma linha de H e calcula mensagens usando os outros bits daquela linha.
3. Com $0 \rightarrow -1$ e $1 \rightarrow +1$, um check de grau d exige produto de sinais $(-1)^d$.
4. Sinal da mensagem: produto exigido pela paridade vezes os sinais dos outros bits.
5. Magnitude da mensagem: menor magnitude entre os outros bits.
6. Cada bit soma crença original + mensagens dos checks; repete até $Hx^T = 0$ ou limite de iterações.

Exemplo da figura da esquerda: ao calcular a mensagem para $r[0]$, considere apenas $r[1..3]$.

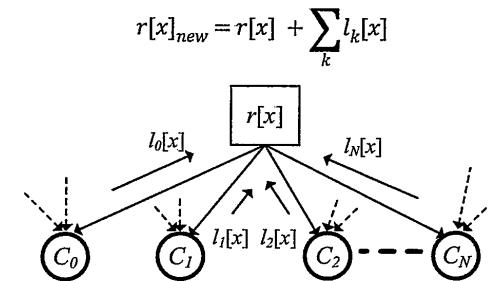
Sinal: $(-) \cdot (-) \cdot (+) = +$. Magnitude: $\min(1.0, 0.9, 1.2) = 0.9$. Logo $l[0] = +0.9$.

$$r = [r[0] \ r[1] \ r[2] \ r[3]] = [-0.1 \ -1.0 \ -0.9 \ 1.2]$$



Schwarzinger, Figure 5-90: Single Parity Check Node.

$$\begin{aligned}l[0] &= 0.9 \\l[1] &= 0.1 \\l[2] &= 0.1 \\l[3] &= -0.1\end{aligned}$$

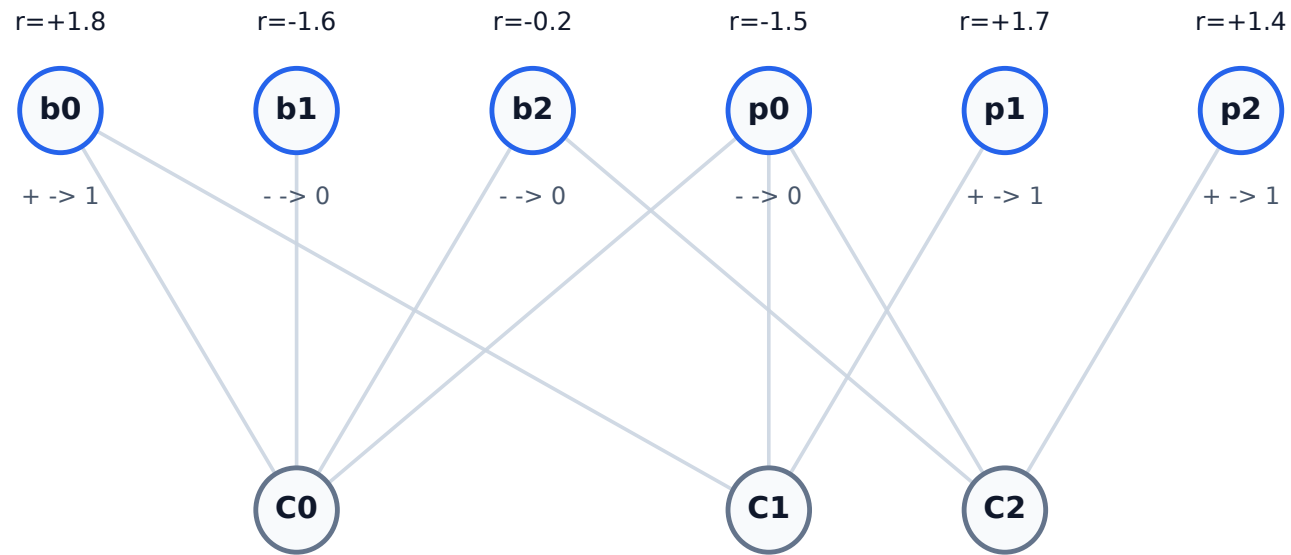


Schwarzinger, Figure 5-91: Updating Intrinsic Belief.

$$r[x]_{new} = r[x] + \sum_k l_k[x]$$

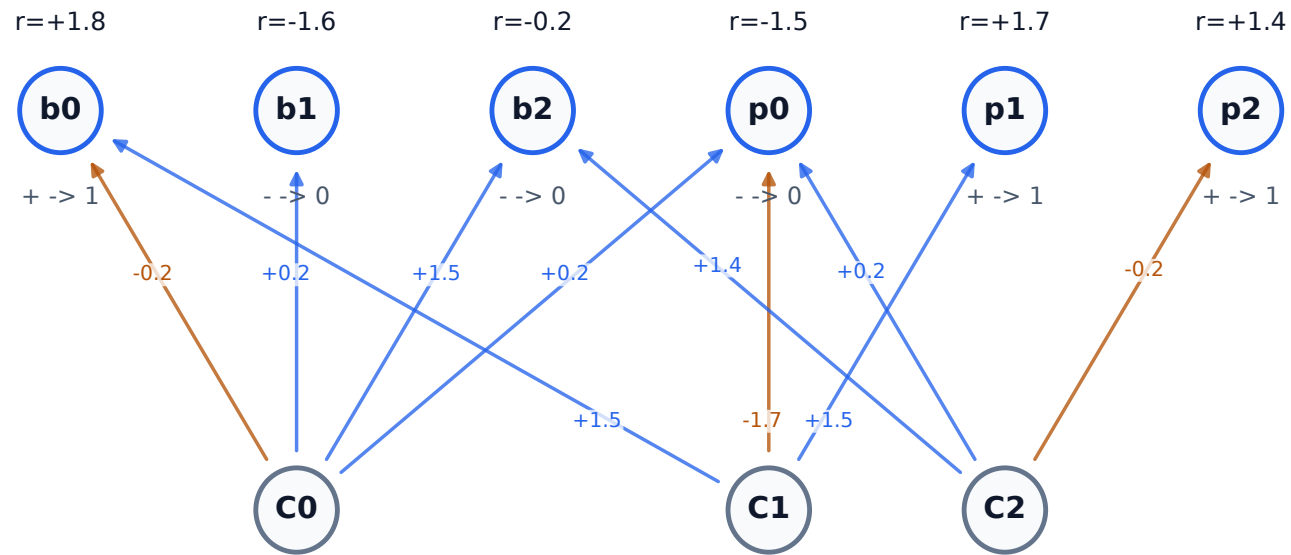
Passo 1: crenças iniciais recebidas

O decoder começa apenas com LLRs dos bits; ele ainda não sabe qual posição está errada.



Passo 2: checks enviam mensagens extrínsecas

Cada check envia uma mensagem para cada bit vizinho, sempre usando os outros bits da mesma linha.



Todas as arestas

recebem uma mensagem
check $\rightarrow$ bit.

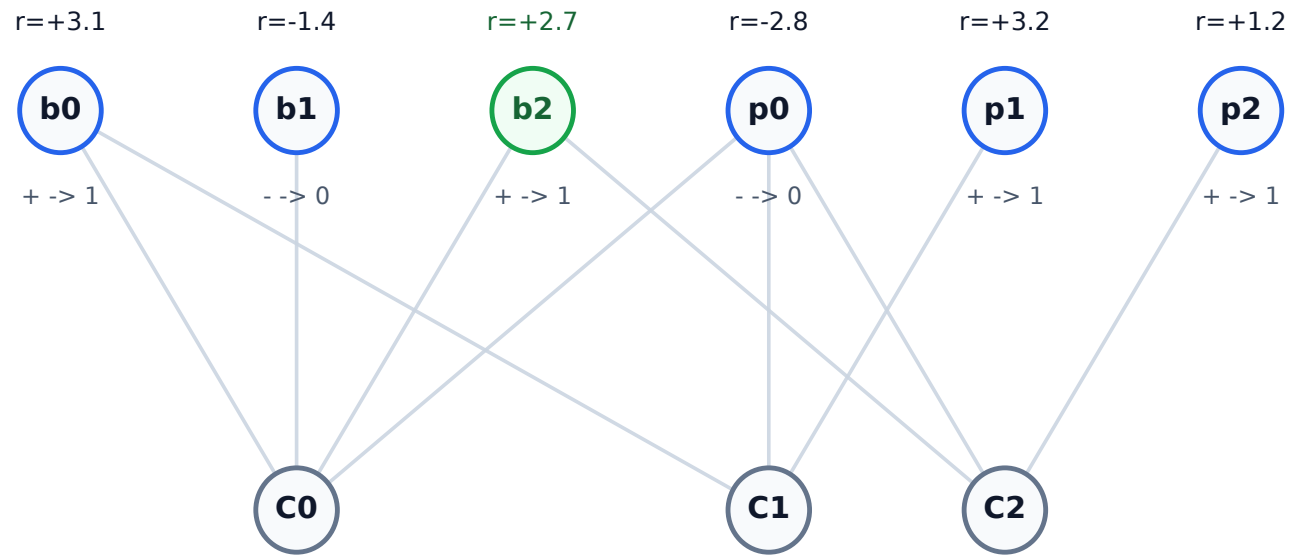
Azul: mensagem positiva

Marrom: mensagem negativa

A atualização do bit
acontece só no
próximo passo.

Passo 3: bits somam mensagens e decidem

Depois de receber todas as mensagens, cada bit atualiza seu LLR e toma uma nova decisão hard.



Atualização

$$r_{\text{new}} = r + \text{soma}(l)$$

Apenas um nó muda de decisão hard.

No grafo, isso faz todos os checks ficarem consistentes.

Foi esse bit que o canal havia invertido.

Como a matriz H é escolhida?

H é uma escolha de projeto do código, não uma matriz gerada a partir da mensagem.

- Escolhe-se comprimento da palavra-código n , bits úteis k e taxa $R = k/n$.
- Graus dos nós e estrutura do grafo buscam convergência iterativa e poucos ciclos curtos.
- Uma construção estruturada transforma a ideia em matriz implementável; o próximo slide mostra essa estrutura.

No TGN/802.11, as propostas convergiram para códigos estruturados:

- matrizes-semente pequenas H_b ;
- expansão por matrizes de permutação cíclicas;
- várias taxas, comprimentos n e fatores de expansão Z com baixa memória de descrição;
- encoding quase linear;
- a matriz também é escolhida pensando no decoder: LDPC usa passagem iterativa de LLRs no grafo de Tanner; nos documentos isso aparece como BP (*belief propagation*), SPA (*sum-product*) e *layered BP*.

A seleção é validada por simulação, não por uma garantia simples de “corrige até t erros”:

- BER (probabilidade de erro por bit), BLER (por bloco LDPC) e PER (por pacote/quadro) versus SNR em AWGN e canais relevantes;
- tamanhos de pacote, taxas e número de iterações;
- *error floor*, throughput, área, memória e consumo.

LDPC estruturado: de H_b para H

H_b é uma matriz-base pequena. Cada entrada vira um bloco $Z \times Z$ na matriz real H :

—	0	2	—
1	—	0	3
—	2	—	0

→

0	I	P^2	0
P^1	0	I	P^3
0	P^2	0	I

H_b : $m_b = 3$ linhas, $n_b = 4$ colunas

H : $m_b Z$ checks por $n_b Z$ bits

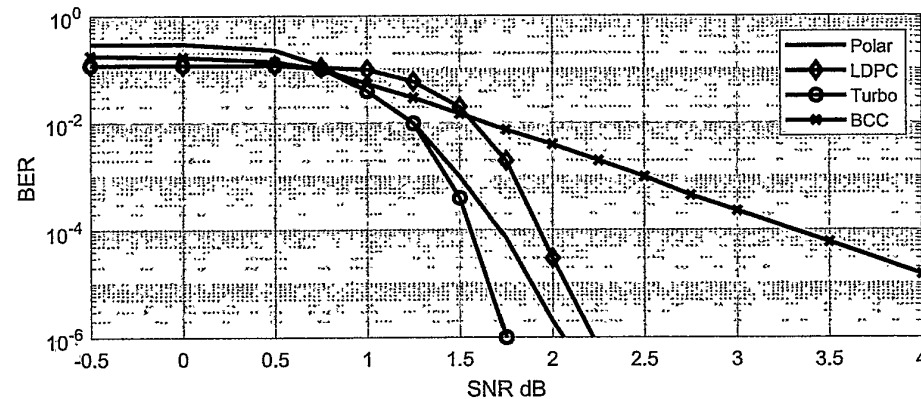
- Entrada “—” em H_b : bloco zero, sem conexões no grafo.
- Entrada $p \geq 0$: identidade cíclica P^p (vide exemplo ao lado); desloca conexões dentro do bloco.
- O tamanho do código aparece nas dimensões: $n = n_b Z$ é o comprimento; se todos os checks são independentes, $k = n - m_b Z$ bits úteis.
- A norma escolhe diferentes matrizes-base, fatores Z e taxas para obter códigos implementáveis em hardware.

Exemplo de bloco, $Z = 4$:

0	1	0	0
0	0	1	0
0	0	0	1
1	0	0	0

P^1 : identidade deslocada

BCC versus LDPC



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 5-92: Rate 1/2 FEC Performance Comparison.

- BCC + Viterbi: bom para mensagens curtas, baixa latência e hardware previsível.
- LDPC: melhor para blocos longos e opera mais perto do limite de Shannon.
- BCC melhora gradualmente com SNR.
- LDPC tem comportamento de “waterfall”: pouco ganho abaixo de certo SNR, queda abrupta de BER acima dele.
- Wi-Fi evoluiu de BCC obrigatório em 802.11a/g para LDPC opcional em gerações posteriores, com uso cada vez mais relevante em taxas altas.

Por que Wi-Fi ficou com LDPC?

Linha do tempo

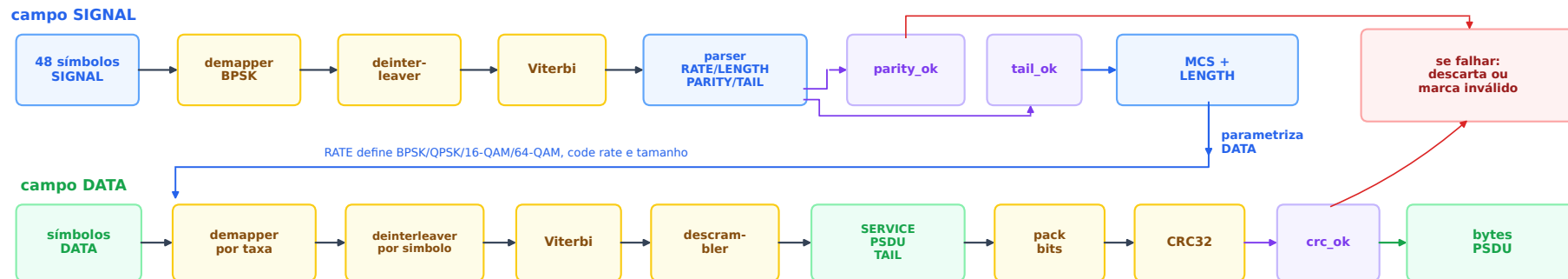
- 1960/1962/1963: Gallager propõe LDPC; a ideia era boa, mas cara demais para o hardware da época.
- 1993: Turbo codes mostram desempenho muito perto do limite de Shannon e mudam a área.
- 1995/1996: MacKay e Neal mostram LDPC esparsos com desempenho também próximo ao limite.
- 2009: Polar codes aparecem depois das escolhas técnicas do 802.11n.

Decisão prática no TGN

- As atas do TGN tratavam Turbo e LDPC como alternativas de desempenho parecido em primeira ordem; a curva anterior não decidia o padrão sozinha.
- A latência era concreta: 1600 bits a 240 Mb/s já levam 6,6 μ s só para chegar ao receptor; sobrava um orçamento pequeno, da ordem de 1 μ s, para o FEC.
- Turbo foi proposto, mas trazia patente fundamental ativa, interleaver e troca iterativa de informação extrínseca; o LDPC estruturado favorecia pacotes e hardware: matriz-semente pequena, expansão cíclica, baixo armazenamento, encoding quase linear e *layered BP*.
- Resultado: 802.11n adotou LDPC como opção; Turbo não entrou no Wi-Fi.

Fontes: Gallager, *Low-Density Parity-Check Codes* Berrou et al., ICC 1993 MacKay e Neal, 1996 US 5,446,747 IEEE 802.11-04/0902r0 IEEE 802.11-04/1362r0 Arikan, 2009.

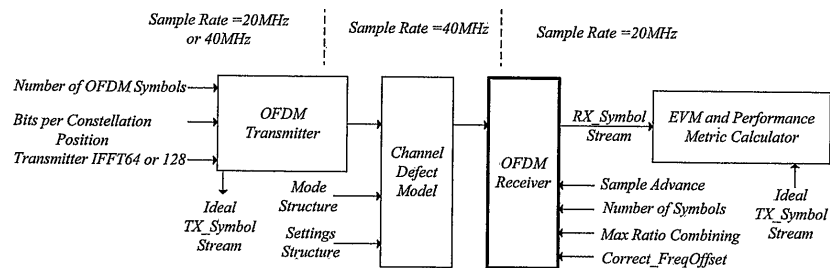
Dos símbolos aos bytes



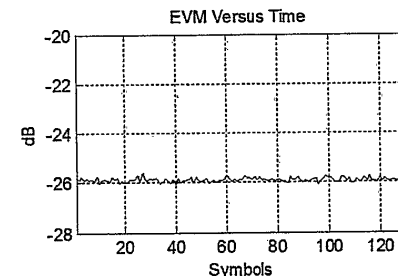
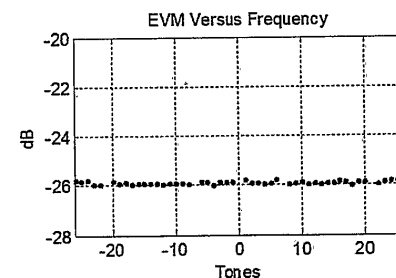
SIGNAL escolhe como DATA será interpretado; DATA produz bytes, mas CRC apenas valida plausibilidade

- `decode_signal_field()`: sempre BPSK 1/2; descobre taxa e comprimento.
- `decode_data_symbols()`: usa parâmetros do SIGNAL; recupera PSDU.
- O CRC não corrige; ele confirma se a sequência corrigida pela FEC é plausível.

Depuração: EVM no tempo e na frequência



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-62: Code Flow of OFDM Transceiver.



Schwarzinger, Digital Signal Processing in Modern Communication Systems, Figure 7-52: EVM Performance at 25 dB SNR.

Use os gráficos como diagnóstico:

- Ruído branco: EVM quase plano em frequência e tempo.
- Multi-caminho: EVM varia com a subportadora.
- Fase residual: EVM piora com o tempo ou constelação gira.
- Timing jitter: tons externos sofrem mais.
- Erro de deinterleaver/Viterbi: constelação pode parecer boa, mas bits falham.

Checklist mental do receptor 802.11

Para cada quadro recebido, pergunte:

1. O pacote foi detectado pela periodicidade da STS?
2. A frequência foi corrigida antes da FFT?
3. A janela FFT está dentro da região segura do GI?
4. O canal estimado pela LTS faz sentido em magnitude e fase?
5. Os pilotos estão corrigindo fase comum e rampa de timing?
6. Os LLRs preservam confiança para o Viterbi?
7. Deinterleaver, descrambler, cauda e CRC concordam?

Esse é o caminho completo do Wi-Fi baseado em SDR: RF vira amostras, amostras viram tons, tons viram símbolos, símbolos viram LLRs/crenças, crenças viram bits, bits viram quadro.